성결대학교

컴퓨터공학과



졸업작품 최종보고서

MARU

외국인을 위한 한국어 학습 앱

Morpheme-based · Adaptive · Roleplay · Upgrade Your Expression

구분	2026년 졸업작품 · 설계 프로젝트 경진대회
지도 교수	한경수 교수님
팀 명	마루
팀 원	하도윤
학번	20200679
제출일	2026년 10월

목 차

1. 서론 6

1.1 개발 배경 6

1.1.1 글로벌 한국어 학습 수요의 급증 6

1.1.2 한국어의 구조적 학습 난이도 6

1.1.3 기존 한국어 학습 앱의 구조적 한계 7

1.2 연구 목적 및 개발 목표 7

1.3 학습 설계 철학 — 토끼와 거북이 8

1.4 구현 범위 요약 9

1.5 보고서 구성 9

2. 관련 연구 및 기술 검토 11

2.1 컴퓨터 보조 언어 학습(CALL)의 발전과 모바일 앱의 한계 11

2.1.1 CALL의 전개 11

2.1.2 기존 모바일 앱의 구조적 한계 11

2.2 간격 반복 시스템(SRS)과 망각곡선 이론 12

2.2.1 에빙하우스 망각곡선과 분산 학습 12

2.2.2 SM-2 알고리즘과 한계 12

2.2.3 FSRS 알고리즘의 등장 12

2.3 교착어로서의 한국어와 형태소 기반 학습 13

2.3.1 한국어의 교착어적 특성과 제2언어 습득 13

2.3.2 한국어 형태소 분석기 Kiwi(kiwipiepy) 13

2.4 LLM 기반 대화형 언어 학습 14

2.4.1 LLM의 언어 교육 적용 연구 14

2.4.2 역할극(Role-play) 기반 언어 학습 14

2.4.3 AI 캐싱과 교육적 일관성 14

2.5 선행 연구와 MARU의 차별성 15

3. 시스템 설계 16

3.1 전체 시스템 아키텍처 16

3.2 기술 스택 16

3.3 데이터베이스 설계 17
3.3.1 사용자 도메인 (users · user_stats · user_progress) 18
3.3.2 레슨 도메인 (lessons) 19
3.3.3 단어 도메인 (word_categories · words · fsrs_progress) 20
3.3.4 Mission Chat 도메인 (mission_clearances) 21
3.3.5 AI 캐시 도메인 (ai_cache) 22
3.3.6 TTS 캐시 도메인 (tts_cache) 22
3.3.7 전체 ERD 요약 22
3.4 인증 설계 (Google · Apple OAuth + JWT) 23
3.5 API 설계 24
3.6 오류 처리와 보안 26
4. 핵심 기능 구현 28
4.1 Korean Lesson — 형태소 기반 학습 28
4.1.1 설계 배경: 교착어를 어떻게 가르칠 것인가 28
4.1.2 목표 문법 형태소 분리 알고리즘 28
4.1.3 형태소 드래그앤드롭 퀴즈 생성 파이프라인 (kiwi-generator) 29
4.1.4 Tap-to-Translate: 형태소 의미 주입 파이프라인 30
4.1.5 백엔드 주도(Backend-Driven) 동적 렌더링과 JSONB 설계 32
4.1.6 점수·별·이어하기 32
4.1.7 서버 TTS — 자연스러운 한국어 음성 33
4.2 Vocabulary System — 적응형 단어 학습 33
4.2.1 AI 데이터 파이프라인 (단어 데이터 자동 구축) 33
4.2.2 안정성과 비용을 위한 두 가지 설계 34
4.2.3 FSRS 기반 간격 반복(Spaced Repetition) 알고리즘 35
4.2.4 일일 복습(Daily Word Review)과 매칭 게임 36
4.2.5 평가 무결성과 다음 간격 미리보기 36
4.3 Mission Chat — 역할극 미션 대화 37
4.3.1 4단계 LLM 호출 구조 37
4.3.2 비동기 병렬 대화 처리 (토끼·거북이 역할 분리) 37
4.3.3 3-Zone 턴 제어 시스템 38
4.3.4 사용 프롬프트 39
4.3.5 목표 달성 판정과 피드백 리포트 40

4.3.6 레드카드·옐로카드와 토끼 변신 로딩 40
4.3.7 난이도 설계 — Easy·Normal·Hard 41
4.4 Language Lab — 실험형 인터랙티브 학습 42
4.4.1 Hangeul Lab — 유니코드 한글 조합 42
4.4.2 Sentence Lab — AI 기반 문장 변형 실험 43
4.4.3 AI 캐싱 아키텍처와 정렬 기반 캐시 키 43
4.4.4 입력 검증과 오류 처리 44
4.4.5 화면 설계 — 모드 구분과 거북이 실험 컨셉 44
4.5 캐릭터 시스템 — 토끼와 거북이 45
5. 결과 및 시연 49
5.1 로그인, 홈 화면, 학습 통계 49
5.2 Korean Lesson — 한글 레슨과 형태소 조립 49
5.3 Vocabulary System — FSRS 간격 반복 단어 학습 51
5.4 Mission Chat — LLM 역할극 미션 대화 52
5.5 Language Lab — 한글 조합과 Sentence Lab 54
5.6 운영 배포와 실기기 동작 55
5.7 구현 결과 종합 55
6. 개발 과정과 검증 57
6.1 병렬 세션 개발 방법론 57
6.2 검증 원칙 — 주장과 동작의 일치 58
6.3 사용자 피드백 반영 58
6.4 자동 테스트와 통합 59
6.5 배포와 데이터 이관 60
7. 결론 및 향후 계획 61
7.1 개발 성과 요약 61
7.2 현재 한계점 62
7.2.1 Tap-to-Translate 정적 규칙 문제 62
7.2.2 Mission Chat의 난이도 제어와 교정 정확도 62
7.2.3 Korean Lesson 콘텐츠 규모 62
7.2.4 UI/UX 완성도 62

7.2.5 학습 효과 검증과 알고리즘 개인화 63
7.3 향후 개발 계획 63
7.4 맺음말 63
8. 참고 문헌 65

1. 서론

1.1 개발 배경

1.1.1 글로벌 한국어 학습 수요의 급증

2000년대 이후 한국 대중문화는 전 세계 시장에서 전례 없는 파급력을 발휘하고 있다. K-pop, K-드라마, 한국 영화 등 이른바 K-콘텐츠의 확산은 한국어에 대한 직접적인 학습 수요로 이어졌고, 이는 수치로도 명확히 확인된다. 글로벌 한국어 학습 시장은 2024년 기준 약 72억 달러 규모에서 2034년 약 670억 달러 규모로 성장할 것으로 전망된다(Research and Markets, 2024). 관광 측면에서도 2024년 한국을 방문한 외국인이 1,510만 명으로 전년 대비 51.1% 증가하였으며, 교육 분야에서는 해외 한국어 수업 운영 학교 수가 2021년 1,806개에서 2025년 2,777개로 54% 늘었다(대한민국 교육부, MOE). 이처럼 한국어 학습에 대한 전 세계적 관심은 더 이상 일시적 현상이 아니며, 효과적인 한국어 학습 도구에 대한 필요성 또한 그에 비례하여 확대되고 있다.

1.1.2 한국어의 구조적 학습 난이도

그러나 한국어는 외국인 학습자에게 구조적으로 높은 장벽을 가지는 언어이다. 미국 외교관 연수기관(FSI)은 영어 화자 기준 학습 난이도에 따라 언어를 네 범주(Category I~IV)로 분류하는데, 한국어는 가장 어려운 범주인 Category IV로 분류되어 있다. 이는 단순히 어휘량의 문제가 아니라, 한국어가 가진 언어 유형적 특성에서 기인한다.

세계 언어는 크게 교착어(agglutinative), 굴절어(inflectional), 독립어(isolating)로 구분된다. 영어·프랑스어·스페인어·독일어 등 전 세계 인구의 약 70%가 사용하는 언어는 굴절어 또는 독립어에 속한다. 이들 언어에서는 문장의 어순이 의미를 결정하는 핵심 기제이며, 단어의 형태가 비교적 고정적이다. 반면 한국어는 교착어로, 어근에 조사·어미가 순차적으로 결합하여 문법 기능을 나타낸다. '나는 한국어를 공부한다'에서 조사 '는'(주제 표지), '를'(목적격 표지)이 분리되어야 비로소 문장의 구조가 파악된다. 어순이 바뀌어도 조사가 의미를 유지하며, 동사가 문장 맨 끝에 위치하는 SOV 구조는 굴절어 또는 독립어 화자에게 직관적으로 이해되지 않는다.

더욱이 한국어의 교착 특성은 번역기조차 해결하지 못하는 문제를 낳는다. 예컨대 '이름은'을 영어로 번역하면 Google 번역기와 DeepL 모두 'The name is'로 오역한다. 조사 '은'의 주제 표지 기능이 동사로 오인되는 것이다. 단어를 통째로 이해하는 한 조사의 존재 자체를 인지할 수 없고, 이것이 한국어 학습자가 단어는 알아도 문장 구성에서 실패하는 근본 원인이다.

1.1.3 기존 한국어 학습 앱의 구조적 한계

이러한 수요와 난이도 사이의 간극을 메우기 위해 다양한 한국어 학습 앱이 등장하였으나, 시장을 선도하는 앱들도 교착어로서의 한국어 특성을 학습 설계에 충분히 반영하지 못하고 있다. 기존 앱들의 구조적 문제는 세 가지로 정리된다.

한계	내용
암기 중심 학습	대부분의 앱은 객관식 문제와 단어·문법 암기 중심으로 구성되어 있다. 학습자는 '이것이 맞다/틀리다'를 판단하는 수동적 역할에 머물며, 스스로 문장을 조립하거나 문법 구조를 체험할 기회가 없다.
상호작용 부족	학습자가 직접 문장을 구성하거나 실제 대화 맥락에서 한국어를 응용하는 기회가 극도로 제한된다. 언어 습득에 필수적인 생산적 사용(productive use) 기회가 없어 실제 의사소통 능력으로 이어지지 않는다.
체계적 학습 부족	한글의 자모 조합 원리, 조사·어미의 문법 기능, 문장 구조의 단계적 이해를 설명하는 체계적 학습 시스템이 부재하다. 덩어리 암기에 그쳐 학습자는 응용 상황에서 쉽게 막힌다.

표 1-1. 기존 한국어 학습 앱의 구조적 한계

이 세 가지 한계는 공통적으로 하나의 문제로 수렴한다. 단어를 외우는 것과 조사·어미의 기능을 이해하며 문장을 조립하는 것은 근본적으로 다른 인지 과정이다. 기존 앱들은 전자에 집중하고 후자를 간과한다.

1.2 연구 목적 및 개발 목표.

본 프로젝트는 위에서 제시한 문제 인식을 출발점으로, 외국인 학습자가 한국어의 구조를 직접 체험하며 익히는 모바일 학습 앱 MARU를 설계·개발하는 것을 목적으로 한다. '마루'는 한국 전통 가옥에서 사람들이 모여 생활하는 중심 공간을 뜻하는 단어이자, 동시에 산의 정상을 의미하기도 한다. 전 세계 한국어 학습자들이 모여드는 플랫폼으로서, 그리고 각자의 학습 정상을 향한 여정을 함께하는 동반자로서 이 이름을 붙였다.

핵심 개발 목표는 기존 앱의 세 가지 한계를 각각 기술적 설계로 직접 해소하는 것이다. 목표를 MARU라는 이름의 두문자로 구조화하면 다음과 같다.

두문자	목표	해소하는 한계
M — Morpheme-based Learning	형태소 단위 드래그앤드롭 학습으로 교착어 구조를 손끝으로 체험하게 한다.	암기 중심 학습 탈피 + 체계적 구조 이해
A — Adaptive Vocabulary	FSRS-4.5 망각곡선 알고리즘 기반 개인 맞춤형 간격 반복으로 단어를 기억하게 한다.	암기 중심에서 기억 시스템으로 전환
R — Roleplay Mission	LLM 기반 역할극 미션 대화로 실제 맥락에서 한국어를 사용하는 경험을 제공한다.	상호작용 부족 해소
U — Upgrade Your Expression	한글 조합 실험과 AI 문법 변형 실험으로 표현을 직접 설계하고 확장하게 한다.	체계적 학습 부족 + 생산적 사용 기회 제공

표 1-2. MARU의 개발 목표

이 네 가지 목표는 단순히 기능을 나열한 것이 아니라, 교착어인 한국어의 언어학적 특성을 기술적 설계로 직접 번역한다는 일관된 철학 아래 연결되어 있다. 형태소 분석기를 목표 문법 분해에, 망각곡선 수식을 복습 스케줄에, LLM을 교육학적 대화 궤도로, 유니코드 한글 조합 규칙을 직접 체험 도구로 변환한 것이 MARU의 핵심 설계 방식이다.

1.3 학습 설계 철학 — 토끼와 거북이

MARU 의 학습 설계는 전래동화 '토끼와 거북이'에서 모티프를 가져온다. 이는 단순한 은유가 아니라, 언어 습득 이론에서 말하는 유창성(fluency)과 정확성(accuracy) 의 균형을 학습 경험 수준에서 구현한 것이다.

토끼 모드는 빠르고 직관적인 체험을 지향한다. 의미 덩어리(어절) 단위로 문장을 조립하고, 일단 해보는 학습을 먼저 경험하게 한다. 학습자는 한국어 문장의 전체 흐름을 먼저 체험함으로써 언어에 대한 친숙함과 자신감을 얻는다.

거북이 모드는 차분하고 깊은 이해를 지향한다. 같은 문장을 다시 형태소 단위로 분해하되, 해당 레슨이 가르치려는 핵심 조사·어미만 선택적으로 분리한다. 틀리면 구조적 설명을 통해 코칭하여 정확성을 보강한다.

이 두 모드는 모든 핵심 기능에 일관되게 적용된다. Korean Lesson 의 드래그앤드롭 퀴즈, Mission Chat 의 토끼(연기)·거북이(채점) 역할 분리, Vocabulary System 의 덱 학습과 Daily Review, Language

Lab 의 즉각 체험과 AI 문법 실험이 모두 이 철학 위에 설계되어 있다. 두 모드는 서로 경쟁하는 방식이 아니라 하나의 학습 사이클 안에서 순서대로 작동하며, 직관과 구조 이해를 함께 쌓아가도록 유도한다. 이 철학은 앱 안에서 토끼와 거북이 캐릭터로 구현되어 있다. 토끼는 체험과 연기(정답 반응, 미션 속 역할 연기)를, 거북이는 분석과 코칭(오답 힌트, 문법 교정)을 맡아 학습 이벤트에 실시간으로 반응한다(4.5절).

1.4 구현 범위 요약

MARU는 로그인부터 레슨, 단어 복습, 역할극 대화, 실험실까지 전 흐름이 이어지는 앱으로 구현되었으며, 운영 서버에 배포되어 실제 iPhone에서 동작한다. 구현 범위를 요약하면 다음과 같다. 본 보고서의 수치는 모두 서버 로그와 자동 테스트로 확인한 값만 사용하였다.

영역	구현 내용	절
Korean Lesson	한글 12레슨(자모·음절 80항목) + 문법 3레슨(은/는, 이에요/예요, 을/를). 레슨 목표 문법만 형태소로 분해하는 2단계 조립 문제, 정답률 점수·별·이어하기, 서버 TTS	4.1
Vocabulary	국립국어원 학습용 어휘 기반 5,561단어·14개 주제 덱·30단어 레슨. FSRS-4.5 표준 공식으로 복습일 산출, 오늘의 복습, 짝맞추기(라운드당 5쌍)	4.2
Mission Chat	토끼(연기)·거북이(채점) 병렬 호출(한 턴 약 1.5~2초), 난이도 3단계, 서버 강제 3-Zone, 목표 달성 판정과 피드백 리포트, 레드·옐로카드	4.3
Language Lab	기기 내 한글 조합(Hangeul Lab), AI(Gemini) 기반 문장 변형(Sentence Lab: Explore/Combine), DB 캐시(HIT 1~2ms / MISS 3~5초)	4.4
캐릭터	토끼·거북이 이미지 21장 + 코드 모션, 4개 기능 21개 화면 지점과 홈에서 학습 이벤트에 반응	4.5
공통	Google·Apple 소셜 로그인 + JWT, 공통 오류 규칙, API 22개, Railway 운영 배포, iPhone 실기기 동작	3장, 5.6
개발·검증	기능별 병렬 세션 개발, 사용자 피드백 4라운드 반영, 자동 테스트(서버 135개·앱 83개 통과)	6장

표 1-3. MARU 구현 범위 요약

1.5 보고서 구성

본 보고서는 다음과 같이 구성된다.

장	제목	내용
1장	서론	개발 배경, 연구 목적, 학습 설계 철학, 구현 범위 요약
2장	관련 연구 및 기술 검토	기존 언어 학습 앱 분석, 형태소 분석·간격 반복·LLM 활용 관련 기술 검토
3장	시스템 설계	전체 아키텍처, 기술 스택, 데이터베이스 스키마, 인증, API, 오류 처리와 보안
4장	핵심 기능 구현	Korean Lesson·Vocabulary System·Mission Chat·Language Lab·캐릭터 시스템의 설계와 구현 상세
5장	결과 및 시연	기능별 구현 결과 화면, 운영 배포와 실기기 동작
6장	개발 과정과 검증	병렬 세션 개발 방법론, 검증 원칙, 사용자 피드백 반영, 자동 테스트, 배포·데이터 이관
7장	결론 및 향후 계획	개발 성과 정리, 남은 한계점, 확장 방향
8장	참고 문헌	논문, 기술 문서, 오픈소스

표 1-4. 보고서 구성

2. 관련 연구 및 기술 검토

본 장에서는 MARU의 설계 근거가 되는 핵심 이론 및 기술 선행 연구를 검토한다. 크게 네 가지 영역인 (1) 컴퓨터 보조 언어 학습(CALL)의 발전, (2) 간격 반복 시스템과 망각곡선 이론, (3) 교착어로서의 한국어와 형태소 기반 학습, (4) LLM 기반 대화형 언어 학습으로 나누어 검토하고, 각 연구가 MARU 설계에 어떻게 반영되었는지를 연결하여 기술한다.

2.1 컴퓨터 보조 언어 학습(CALL)의 발전과 모바일 앱의 한계

2.1.1 CALL 의 전개

컴퓨터 보조 언어 학습(Computer-Assisted Language Learning, CALL)은 1960년대 단순 드릴 연습 프로그램에서 출발하여, 2000년대 이후 인터넷 보급과 함께 상호작용 중심으로 진화하였다. 2010년대 스마트폰 보급 이후에는 모바일 앱 형태의 CALL이 폭발적으로 성장하였으며, Duolingo(2011), Drops(2015) 등이 전 세계 수억 명의 사용자를 확보하였다. 이 시기 CALL 연구는 주로 게임화(gamification)와 동기 부여 측면에 집중되었다.

Shadiev et al.(2021)은 Duolingo 관련 문헌을 체계적으로 검토한 결과, 연구의 대부분이 학습 설계나 게임화 요소 분석에 집중되어 있었으며 실제 학습 결과물을 측정한 연구는 소수에 불과하다고 지적하였다 특히 게임화가 동기를 높이는 효과는 인정되나, 경쟁적 순위표(leaderboard)나 연속 달성(streak) 요소가 오히려 학습 목표 자체를 왜곡하여 게임 점수 획득에 집중하게 만드는 역효과도 보고되었다

2.1.2 기존 모바일 앱의 구조적 한계

Duolingo를 비롯한 주요 언어 학습 앱의 핵심 문제는 단어·문법의 수용적 인식(receptive recognition) 에 특화되어 있다는 것이다. 즉, 주어진 선택지 중에서 맞는 것을 고르는 연습은 충분하지만, 학습자가 스스로 문장을 조립하거나 실제 의사소통 상황에서 언어를 생산적으로 사용(productive use) 하는 기회는 극히 제한적이다

이는 한국어와 같은 교착어 학습에서 특히 치명적인 한계로 작용한다. 한국어는 조사·어미가 어근에 결합하여 문법 기능을 나타내므로, 덩어리 암기만으로는 새로운 문장 생성 능력이 발달하지 않는다. 학습자는 단어를 알아도 올바른 조사를 선택하여 문장을 조립하는 데 실패하는 경우가 많으며, 이는 수용적 연습만으로는 교착어의 구조 이해가 이루어지지 않음을 시사한다.

MARU는 이 한계를 형태소 단위 드래그앤드롭 퀴즈(Korean Lesson)와 LLM 기반 역할극 미션(Mission

Chat)을 통해 직접 해소한다. 학습자는 단순히 선택지를 고르는 것이 아니라 문장의 구성 요소를 직접 조립하고, 실제 대화 맥락에서 언어를 생산적으로 사용하는 경험을 얻는다.

2.2 간격 반복 시스템(SRS)과 망각곡선 이론

2.2.1 에빙하우스 망각곡선과 분산 학습

독일 심리학자 헤르만 에빙하우스(Hermann Ebbinghaus)는 1885년 자기 자신을 피험자로 삼아 기억 유지 실험을 수행하고, 학습 후 시간이 지남에 따라 기억이 지수적으로 감소하는 망각곡선(Forgetting Curve) 을 발견하였다. 그의 연구는 학습 직후 24시간 내에 학습 내용의 약 67%가 망각된다는 것을 보였으며, 반복 복습이 망각 속도를 늦추고 장기 기억으로의 전환을 촉진함을 최초로 실증하였다.

이후 분산 학습(distributed practice)의 효과는 수많은 연구를 통해 재확인되었다. 254개 연구를 메타분석한 결과, 집중 학습(massed practice)보다 분산 학습이 10~30% 높은 장기 기억 유지율을 보임이 밝혀졌다. 이 원리를 실용적 알고리즘으로 구현한 것이 간격 반복 시스템(Spaced Repetition System, SRS)이다.

2.2.2 SM-2 알고리즘과 한계

폴란드의 Piotr Woźniak은 1987년 에빙하우스의 이론을 컴퓨터 알고리즘으로 구현한 SM-2 알고리즘을 개발하였다. SM-2는 사용자의 복습 결과(1~5점 평가)에 따라 다음 복습까지의 간격(interval)과 난이도 계수(ease factor)를 동적으로 조정한다 SM-2는 Anki, Mnemosyne 등 주요 플래시카드 앱에서 여전히 사용되고 있으나, 고정된 수식 구조로 인해 개인별 기억 특성의 다양성을 충분히 반영하지 못하는 한계가 있다.

2.2.3 FSRS 알고리즘의 등장

Ye et al.(2022)은 ACM SIGKDD 2022에서 FSRS(Free Spaced Repetition Scheduler)의 기반이 되는 알고리즘을 제안하였다. FSRS는 기억을 안정성(Stability), 난이도(Difficulty), 회상 확률(Retrievability)의 세 가지 상태로 모델링하고, 학습 이력에 따라 이 세 변수를 동적으로 갱신하는 방식으로 다음 복습 일정을 최적화한다

FSRS는 SM-2 대비 약 12.6% 향상된 기억 예측 정확도를 보고하였으며, 오픈소스로 공개되어 Anki 플러그인 등으로 실용화되었다. 특히 FSRS는 복습 결과를 Again / Hard / Good / Easy 네 단계로 구분하고, 상태를 New → Learning → Review → Relearning으로 전이시키는 상태 기계(state machine) 구조를 채택한다.

MARU는 FSRS를 어휘 학습 시스템(Vocabulary System)의 스케줄링 엔진으로 채택하였다. 구현에는 공개된 FSRS-4.5 표준 공식과 기본 파라미터(w0~w16)를 사용한다. FsrsAlgorithm.java는 단어별 안정성(stability)·난이도(difficulty)와 경과일에 따른 인출 가능성(retrievability)을 계산하고, 목표 기억률 0.9에 도달하는 시점을 다음 복습일(next_review_date)로 정한다(4.2.3절).

2.3 교착어로서의 한국어와 형태소 기반 학습

2.3.1 한국어의 교착어적 특성과 제 2 언어 습득

한국어는 형태론적으로 교착어(agglutinative language)에 속하며, 어근에 복수의 기능 형태소(조사, 어미)가 순차적으로 결합하여 문법 정보를 나타낸다. 이 특성은 영어·프랑스어 등 굴절어 또는 독립어를 모어로 하는 학습자에게 근본적으로 다른 문장 처리 전략을 요구한다.

Kim(2024)은 L2 한국어 학습자의 조사 오류를 분석하여, 조사가 한국어 제2언어 습득에서 가장 높은 오류율을 보이는 문법 항목임을 확인하였다 이 연구는 학습자가 단어 수준의 어휘를 습득하더라도 조사·어미의 기능을 독립적 단위로 인식하지 않으면 문장 생성 능력으로 이어지지 않음을 보였다.

Park & Kim(2024)은 L2 한국어 말뭉치의 형태소 풍부도(morphological richness)를 분석하여, 내용 형태소(content morpheme)는 초급~중급 단계에서 빠르게 발달하지만 기능 형태소(조사, 어미)는 상급 단계까지도 오류가 지속됨을 밝혔다 이는 기능 형태소를 명시적으로 분리하여 학습하는 접근법의 교육적 필요성을 시사한다.

2.3.2 한국어 형태소 분석기 Kiwi(kiwipiepy)

자동 형태소 분석은 텍스트를 형태소 단위로 분리하고 품사를 태깅하는 자연어 처리 기술이다. 한국어 형태소 분석은 교착어 특성상 어절 경계와 형태소 경계가 일치하지 않아 영어보다 훨씬 복잡한 처리가 요구된다.

본 프로젝트는 한국어 형태소 분석기로 Kiwi를 채택하였다. Kiwi는 통계적 언어 모델과 Skip-Bigram을 결합하여 고속·고정밀 형태소 분석을 구현한 오픈소스 분석기로, Python 바인딩인 kiwipiepy를 통해 서비스된다

MARU는 kiwi-generator 파이프라인에서 kiwipiepy를 핵심 엔진으로 사용한다. 이 파이프라인은 서비스 안에서 실시간으로 동작하는 것이 아니라 개발자가 실행하는 오프라인 콘텐츠 생성 도구이다. curriculum.csv에 정의된 문장과 학습 목표 품사(Target_POS)를 입력받아, Kiwi가 형태소를 분석하고 목표 문법 형태소만 분리한 2단계 조립 문제 데이터를 생성하며, 결과는 재실행 가능한 SQL 패치로

데이터베이스에 반영된다. Tap-to-Translate 기능을 위한 형태소 청크 주입에도 동일하게 적용된다.

2.4 LLM 기반 대화형 언어 학습

2.4.1 LLM 의 언어 교육 적용 연구

대형 언어 모델(Large Language Model, LLM)의 급속한 발전은 컴퓨터 보조 언어 학습 분야에 새로운 가능성을 열었다. 특히 GPT-4의 등장 이후 LLM을 대화 파트너 또는 교육 에이전트로 활용하는 연구가 급증하였다.

Hwang et al.(2024)은 LLM 기반 챗봇을 영어 학습에 적용한 연구를 분석하여, 즉각적 피드백, 개인화된 대화, 학습 자율성 강화 측면에서 유의미한 효과가 있음을 확인하였다 특히 LLM은 실제 원어민과의 대화에서 오는 심리적 부담 없이 반복 연습이 가능하다는 점에서 학습 동기 유지에 긍정적으로 작용한다.

반면 연구들은 LLM의 한계도 지적한다. LLM은 미묘한 문법 오류를 간과하거나 유창성을 정확성보다 우선하는 경향이 있어, 오류 수정 없이 잘못된 표현을 강화할 위험이 있다. 또한 문화적 맥락이나 화용론적 적절성을 판단하는 능력이 인간 교사 대비 제한적이다.

2.4.2 역할극(Role-play) 기반 언어 학습

역할극은 제2언어 교육에서 오래전부터 활용되어온 효과적인 교수법 중 하나이다. 학습자는 특정 사회적 역할과 시나리오 안에서 목표 언어를 사용해야 하는 상황에 놓임으로써, 언어의 화용론적(pragmatic) 측면, 즉 상황에 맞는 표현 선택 능력을 발달시킨다.

OpenAI(2023)의 GPT-4 기술 보고서는 GPT-4가 복잡한 역할을 일관성 있게 유지하며 대화 맥락을 추적하는 능력이 GPT-3.5 대비 현저히 향상되었음을 보였다 이는 LLM을 역할극 기반 언어 학습에 적용할 때 일관된 시나리오 유지와 맥락 적절 피드백 제공이 가능함을 의미한다.

MARU의 Mission Chat은 이 연구 성과를 실제 학습 설계에 반영한다. GPT-4o를 토끼(연기자)와 거북이(채점자)의 두 역할로 분리하여 병렬 호출함으로써, 유창한 대화 흐름 유지(토끼)와 문법적 정확성 평가(거북이)를 동시에 수행한다. 이 구조는 LLM이 유창성을 우선하는 경향으로 인해 문법 피드백이 약화되는 문제를 역할 분리로 보완한 설계이다.

2.4.3 AI 캐싱과 교육적 일관성

LLM을 교육 서비스에 상시 적용할 때 발생하는 실질적 과제는 응답 지연(latency)과 비용이다. AiLabService의 문법 변형 기능처럼 동일한 입력에 반복 호출이 발생하는 경우, LLM을 매번 호출하

는 것은 비효율적이다.

MARU는 ai_cache 테이블을 설계하여 input_text와 transformation_type의 조합을 유니크 키로 캐싱한다. 특히 Sentence Lab처럼 여러 수식어(modifier)를 조합하는 경우, 입력 순서와 관계없이 동일한 결과를 반환하도록 입력 문장을 정규화(앞뒤·연속 공백 정리)하고 정렬된 수식어 키를 사용하여 결정론적 캐시 히트(deterministic cache hit)를 구현하였다. 이는 LLM 응답의 교육적 일관성을 보장하는 동시에 서비스 비용을 절감한다.

2.5 선행 연구와 MARU 의 차별성

지금까지의 선행 연구와 기술 검토를 바탕으로, MARU의 설계 차별성을 다음과 같이 정리할 수 있다.

선행 연구/기술	한계 또는 적용 갭	MARU의 대응
기존 모바일 CALL 앱 (Duolingo 등)	수용적 인식 중심, 생산적 사용 기회 부족, 교착어 구조 반영 미흡	형태소 드래그앤드롭 + 역할극 미션으로 생산적 사용 기회 제공
SM-2 알고리즘	고정 수식 구조, 개인별 기억 특성 반영 한계	FSRS-4.5 표준 공식 채택으로 안정성·난이도·회상 확률 동적 모델링
Kiwi 형태소 분석기	범용 NLP 도구로 교육 목적 품사 설명 부재	목표 문법 형태소만 분리하는 조립 문제 생성 + TOKEN_SPECIFIC_MAP으로 학습자 친화적 품사 설명 레이어 추가
LLM 언어 학습 적용 연구	유창성 우선으로 문법 피드백 약화, 역할 분리 없음	토끼(연기)·거북이(채점) 역할 분리 구조로 정확성 평가 강화
AI 캐싱 전략	단순 키-값 캐싱으로 조합 입력에 캐시 미스 발생	정렬된 수식어 키로 결정론적 캐시 히트 구현

표 2-1. 선행 연구·기술과 MARU의 대응

MARU는 위의 각 기술을 단순히 적용하는 것을 넘어, 교착어로서의 한국어 구조라는 공통된 언어학적 문제의식 아래 형태소 분석·간격 반복·LLM·유니코드 처리를 하나의 일관된 학습 경험으로 통합한 점에서 기존 연구 및 서비스와 차별화된다.

3. 시스템 설계

본 장에서는 MARU의 전체 시스템 아키텍처, 기술 스택, 데이터베이스 스키마, 인증 설계, API 구조를 기술한다. 4장의 핵심 기능 구현이 '무엇을 어떻게 만들었는가'에 집중한다면, 본 장은 그 구현들이 서로 어떻게 연결되어 동작하고 있는지를 설명한다.

3.1 전체 시스템 아키텍처

MARU는 프론트엔드(Flutter), 백엔드(Spring Boot), 데이터베이스(PostgreSQL)의 3-tier 구조로 설계되었다. 백엔드와 데이터베이스는 Railway 클라우드 플랫폼에 배포되어 운영 주소(https://maru2026-production.up.railway.app)에서 동작하며, GitHub main 브랜치에 반영하면 자동으로 재배포된다. AI 기능은 외부 API(OpenAI GPT-4o 대화·gpt-4o-mini-tts 음성, Google Gemini 2.5 Flash 문법 변형)를 호출하고, 같은 요청의 결과는 DB 캐시(ai_cache, tts_cache)로 재사용한다. 프론트엔드는 iOS와 Android를 동시에 지원하는 Flutter 단일 코드베이스이며, 서버 주소는 빌드 시 지정(--dart-define)한다.

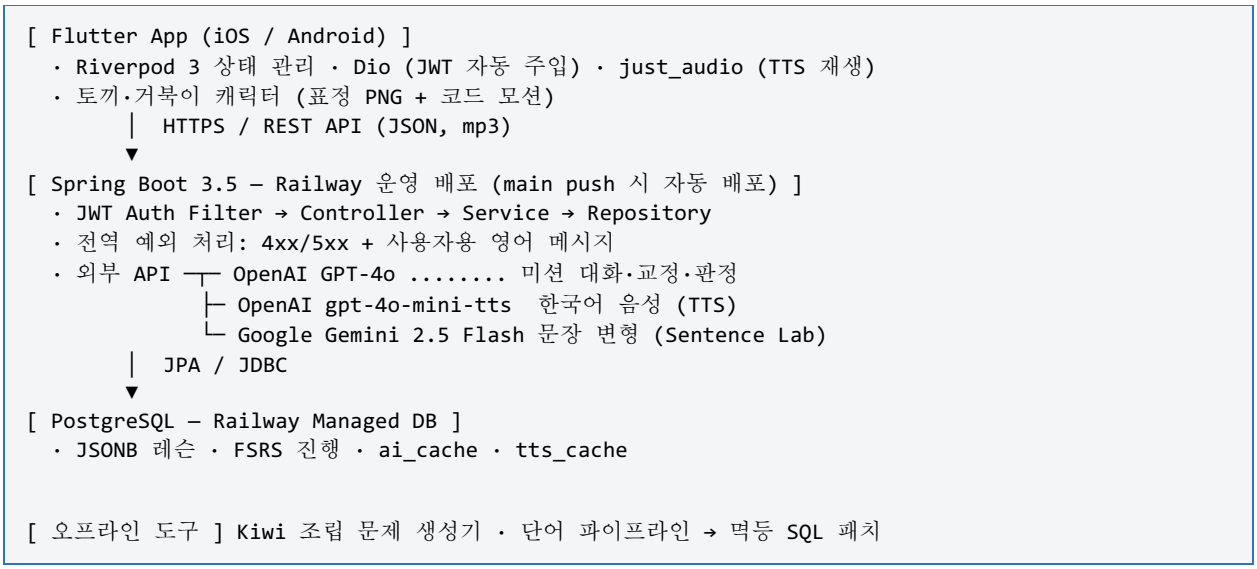


그림 3-1. MARU 전체 시스템 아키텍처 (운영 구성)

3.2 기술 스택

영역	기술	선택 이유
프론트엔드	Flutter 3.x (Dart)	iOS·Android 단일 코드베이스. 드래그앤드롭·카드 플립·캐릭터 모션 등 애니메이션을 네이티브 수준으로 구현
상태 관리	flutter_riverpod ^3.2.1	선언형 의존성 주입·상태 관리. 실패한 요청의 자동 재시도는 끄고 화면별 오류 UI로 처리

영역	기술	선택 이유
HTTP 클라이언트	Dio ^5.9.2	Interceptor로 모든 요청에 JWT Bearer 토큰 자동 주입. 서버 주소는 --dart-define 으로 지정
음성(TTS)	서버: OpenAI gpt-4o-mini-tts 앱: just_audio ^0.10.6 폴백: flutter_tts ^4.2.5	자연스러운 한국어 음성을 서버에서 생성·캐시. 앱은 공통 TtsHelper 한 곳에서 재생, 실패 시 기기 TTS
백엔드 프레임워크	Spring Boot 3.5.11 (Java 17)	RESTful API 설계. Spring Security로 JWT 필터 체인 구성
ORM	Spring Data JPA + Hibernate 6	JSONB 컬럼 자동 직렬화(@JdbcTypeCode). 엔티티-테이블 매핑
데이터베이스	PostgreSQL (Railway)	JSONB 타입 + GIN 인덱스. 레슨 콘텐츠의 이종 구조를 단일 컬럼에 저장
AI (대화)	OpenAI GPT-4o	Mission Chat의 토끼(연기)·거북이(채점) 역할과 목표 달성 판정. JSON 구조 응답
AI (번역·문법)	Google Gemini 2.5 Flash	Vocabulary 데이터 배치 번역(오프라인) 및 Sentence Lab 문장 변형
인증	JWT (jjwt 0.12.5) + Google·Apple OAuth	네이티브 OAuth SDK로 idToken 발급 → 서버에서 서명 검증 → 내부 JWT 발급
형태소 분석	Kiwi (kiwipiepy 0.17)	오프라인 콘텐츠 도구. 어절 경계 계산·품사 태그로 조립 문제와 탭 분석 데이터 생성
캐릭터	AI 생성 표정 PNG 21장 + Flutter 애니메이션	그림(표정·의상)과 몸짓(코드 모션)을 분리해 새 패키지 없이 구현
배포	Railway	Spring Boot JAR + PostgreSQL Managed DB. GitHub main push 시 자동 배포

표 3-1. 기술 스택

3.3 데이터베이스 설계

MARU의 PostgreSQL 데이터베이스는 JPA 엔티티(Entity) 클래스로 정의된 객체 모델을 기반으로 자동 생성된다. Hibernate의 ddl-auto (update) 설정을 통해 엔티티의 변경 사항이 데이터베이스 스키마에 동적으로 반영되며, 단 JSONB 컬럼의 GIN 인덱스는 JPA가 공식적으로 지원하지 않으므로 별도의 SQL 스크립트를 통해 생성한다. 이하에서는 도메인별 테이블 구조와 설계 의도를 기술한다.

3.3.1 사용자 도메인 (users · user_stats · user_progress)

소셜 로그인(Google·Apple) 전용 사용자 테이블이다. 자체 비밀번호를 저장하지 않고, oauth_provider와 oauth_id의 복합 유니크 제약으로 동일 사용자 중복 가입을 방지한다. 내부 PK(id)와 외부 식별자(oauth_id)를 분리하여, 인증 제공자가 바뀌어도 내부 데이터 연관을 유지할 수 있다.

```
CREATE TABLE users (
  id             BIGSERIAL PRIMARY KEY,
  oauth_provider VARCHAR(20)  NOT NULL,   -- 'google' | 'apple'
  oauth_id       VARCHAR(100) NOT NULL,
  email          VARCHAR(100),
  nickname       VARCHAR(50),
  profile_image_url TEXT,
  is_active      BOOLEAN NOT NULL DEFAULT TRUE,
  created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_login     TIMESTAMP,
  UNIQUE (oauth_provider, oauth_id)        -- 중복 가입 방지
);
```

표 3-2. users 테이블

사용자 1명당 하나의 통계 레코드를 가지는 user_stats 테이블이다. user_id를 외래키(FK)이자 유니크(Unique) 컬럼으로 지정하여 User 엔티티와 일대일(1:1) 매핑을 형성한다. 해당 테이블에서는 연속 학습일 수(current_streak_days), 최장 연속 학습일(longest_streak_days), 총 학습 시간(total_study_minutes) 등을 관리하며, last_study_date와 오늘 날짜를 비교하여 스트릭 연속 여부를 판단한다.

```
CREATE TABLE user_stats (
  id                      BIGSERIAL PRIMARY KEY,
  user_id                 BIGINT NOT NULL UNIQUE REFERENCES users(id),
  total_lessons_completed INTEGER DEFAULT 0,
  total_study_minutes     INTEGER DEFAULT 0,
  current_streak_days     INTEGER DEFAULT 0,
  longest_streak_days     INTEGER DEFAULT 0,
  total_stars_earned      INTEGER DEFAULT 0,
  last_study_date         DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

표 3-3. user_stats 테이블

사용자별 레슨 진행 상태를 추적하는 user_progress 테이블이다. 복합 유니크 제약 UNIQUE(user_id, lesson_id)으로 동일 레슨 진행 기록이 중복되지 않는다. status는 세 값(not_started, in_progress, completed)을 가지며, current_step으로 마지막으로 완료한 스텝 인덱스를 보존한다. 앱을 종료하고 재진입해도 이어서 학습할 수 있는 근거가 이 컬럼이다.

```
CREATE TABLE user_progress (
  id               BIGSERIAL PRIMARY KEY,
  user_id          BIGINT NOT NULL REFERENCES users(id),
  lesson_id        VARCHAR(50) NOT NULL,
  status           VARCHAR(20) DEFAULT 'not_started',
  current_step     INTEGER DEFAULT 0,
  score            INTEGER,
  stars_earned     INTEGER,
  attempts         INTEGER DEFAULT 0,
  time_spent_seconds INTEGER DEFAULT 0,
  completed_at     TIMESTAMP,
  UNIQUE (user_id, lesson_id)
);
```

표 3-4. user_progress 테이블

3.3.2 레슨 도메인 (lessons)

레슨 콘텐츠를 저장하는 핵심 테이블이다. 이 content 컬럼의 JSONB 도입이 이 테이블의 핵심 설계 결정이다. 레슨마다 스텝 구조가 완전히 다르기 때문에 (introduction·quiz·agglutinative_quiz·completion 등), 이를 정규화하면 stepType이 추가될 때마다 테이블 마이그레이션이 필요하다. JSONB는 이종 구조를 단일 컬럼에 저장하면서도 GIN 인덱스로 내부 검색을 지원하여 RDB의 안정성과 NoSQL의 유연성을 동시에 확보한다.

```
CREATE TABLE lessons (
  id               BIGSERIAL PRIMARY KEY,
  lesson_id        VARCHAR(50)  NOT NULL UNIQUE,  -- 'u1-l1' 형식
  unit_id          INTEGER      NOT NULL,
  unit_title       VARCHAR(100) NOT NULL,
  order_num        INTEGER      NOT NULL,
  title            VARCHAR(100) NOT NULL,
  description      TEXT,
  difficulty_level INTEGER DEFAULT 1,
  estimated_minutes INTEGER DEFAULT 15,
  content          JSONB        NOT NULL,          -- 스텝 배열 (이종 구조)
  is_published     BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- GIN 인덱스: JSONB 내부 필드 검색 (JPA 미지원 → 수동 생성)
CREATE INDEX idx_lessons_content_gin ON lessons USING GIN (content);
CREATE INDEX idx_lessons_unit ON lessons (unit_id);
```

표 3-5. lessons 테이블 (JSONB content 컬럼 핵심)

content 컬럼에 저장되는 JSON의 구조 예시는 다음과 같다. 각 스텝 객체는 stepType 필드로 타입을 구분하며, Flutter의 StepRenderer가 이 값을 기준으로 위젯을 동적으로 선택한다.

```
{ "steps": [
    { "stepType": "introduction", "contentObj": { "sentences": [...] } },
    { "stepType": "quiz",         "contentObj": { "choices": [...] } },
    { "stepType": "agglutinative_quiz",
      "contentObj": { "sentence": "저는 Sarah입니다.",
                      "elements": [...], "options": [...] } },
    { "stepType": "completion",   "contentObj": {} }
  ]
}
```

표 3-6. lessons.content JSONB 구조 예시

3.3.3 단어 도메인 (word_categories · words · fsrs_progress)

단어장 덱(deck)의 메타 정보를 저장한다. level(예: A1, A2, B1)과 deck_order로 학습 난이도 순서를 제어한다. 14개 테마(음식·감정·장소 등) 카테고리와 30단어 균등 분할이 이 테이블의 데이터로 결정된다.

국립국어원 학습용 어휘 XLS(5,965개 항목)를 AI 파이프라인으로 번역·분류하여 최종 적재한 5,561개의 단어를 저장한다. category_id로 word_categories와 N:1 관계를 맺는다. 복합 유니크 제약 UNIQUE(korean_word, part_of_speech)으로 파이프라인 재실행 시 중복 적재를 방지한다. 동음이의어는 품사(part_of_speech)로 구분된다.

```
CREATE TABLE word_categories (
  id          SERIAL PRIMARY KEY,
  title       VARCHAR(100) NOT NULL,
  level       VARCHAR(20)  NOT NULL,
  deck_order INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE words (
  id                  SERIAL PRIMARY KEY,
  category_id         INTEGER NOT NULL REFERENCES word_categories(id) ON DELETE CASCADE,
  korean_word         VARCHAR(100) NOT NULL,
  primary_meaning     VARCHAR(255) NOT NULL,
  part_of_speech      VARCHAR(50),
  example_sentence    TEXT,
  example_translation TEXT,
  audio_url TEXT,  image_url TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

표 3-7. word_categories · words 테이블

FSRS 알고리즘의 연산 결과를 사용자x단어 단위로 저장하는 테이블이다. 복합 유니크 제약 UNIQUE(user_id, word_id)으로 한 사용자가 동일 단어에 대해 하나의 진행 기록만 가진다. FSRS 알고리즘 지표인 state(학습 상태), stability(기억 안정성), difficulty(인출 난이도), next_review_date(다음

복습일) 등을 저장한다. next_review_date 컬럼에 인덱스(idx_fsrs_next_review)를 두어 Daily Review 쿼리(findDueCardsByUser)의 성능을 확보하였다.

```
CREATE TABLE fsrs_progress (
  id              SERIAL PRIMARY KEY,
  user_id         BIGINT  NOT NULL,
  word_id         INTEGER NOT NULL REFERENCES words(id) ON DELETE CASCADE,
  state           INTEGER NOT NULL DEFAULT 0,   -- 0:NEW 1:LEARNING 2:REVIEW 3:RELEARNING
  stability       DOUBLE PRECISION NOT NULL DEFAULT 0.0,
  difficulty      DOUBLE PRECISION NOT NULL DEFAULT 0.0,
  reps            INTEGER NOT NULL DEFAULT 0,
  lapses          INTEGER NOT NULL DEFAULT 0,
  last_review     TIMESTAMP,
  next_review_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_user_word_progress UNIQUE(user_id, word_id)
);
CREATE INDEX idx_fsrs_next_review ON fsrs_progress (next_review_date);
CREATE INDEX idx_fsrs_user_word   ON fsrs_progress (user_id, word_id);
```

표 3-8. fsrs_progress 테이블

3.3.4 Mission Chat 도메인 (mission_clearances)

미션 클리어 기록을 저장하는 테이블이다. 미션 완료 시 거북이 LLM 이 생성한 good_expressions(좋은 표현), incorrect_expressions(오류 표현), turtle_comment(총평 피드백), next_practice(다음 추천 학습)를 저장한다.

goodExpressions 와 incorrectExpressions 는 List 타입으로, JPA 의 JsonListConverter 를 통해 TEXT 컬럼에 JSON 직렬화하여 저장한다. User 엔티티와 N:1 관계를 맺어 사용자의 클리어 이력 전체를 조회할 수 있다.

```
CREATE TABLE mission_clearances (
  id                  BIGSERIAL PRIMARY KEY,
  user_id             BIGINT NOT NULL REFERENCES users(id),
  mission_title       VARCHAR NOT NULL,
  persona             VARCHAR NOT NULL,
  total_turns         INTEGER NOT NULL,
  good_expressions    TEXT,   -- JSON 직렬화: [{kr:.., en:..}, ...]
  incorrect_expressions TEXT, -- JSON 직렬화
  turtle_comment      TEXT,
  next_practice       VARCHAR,
  cleared_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

표 3-9. mission_clearances 테이블

위 컬럼 외에 미션의 판정 결과와 난이도를 저장하는 nullable 컬럼 네 개(cleared BOOLEAN, result_reason TEXT, goal_condition TEXT, difficulty VARCHAR)가 있다. 스키마는 JPA ddl-auto(update)로 관리하므로, 운영 중 스키마 변경은 컬럼·테이블 추가만 허용하고 이름 변경·삭제는

하지 않는다.

3.3.5 AI 캐시 도메인 (ai_cache)

Language Lab의 Gemini API 응답을 캐싱하는 테이블이다. 복합 유니크 제약(input_text, transformation_type)으로 동일 요청의 중복 캐싱을 방지하며, 같은 컬럼에 복합 인덱스(idx_ai_cache_lookup)를 두어 캐시 조회 성능을 확보한다. transformation_type의 형식은 Explore 모드는 explore:카테고리, Combine 모드는 정렬된 변형자 조합 combine:변형자목록이다. 이 규격화된 키 덕분에 선택 순서가 다른 동일 요청도 동일한 캐시를 재사용한다.

```
CREATE TABLE ai_cache (
  id                  BIGSERIAL PRIMARY KEY,
  input_text          TEXT NOT NULL,
  transformation_type VARCHAR(500) NOT NULL,  -- 'explore:tense' | 'combine:casual,past'
  output_text         TEXT NOT NULL,
  english_translation TEXT,
  explanation         TEXT,
  created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (input_text, transformation_type)
);
CREATE INDEX idx_ai_cache_lookup ON ai_cache (input_text, transformation_type);
```

표 3-10. ai_cache 테이블

3.3.6 TTS 캐시 도메인 (tts_cache)

서버 TTS(4.1.7절)가 생성한 mp3 음성을 저장하는 테이블이다. 캐시 키는 모델·목소리·지시문 버전·정규화된 문장을 이어 붙인 문자열의 SHA-256 해시이며, 같은 문장을 다시 요청하면 OpenAI를 호출하지 않고 저장된 음성을 바로 반환한다. 목소리나 모델을 바꾸면 키가 달라지므로 예전 음성이 섞이지 않는다.

```
CREATE TABLE tts_cache (
  id           BIGSERIAL PRIMARY KEY,
  cache_key    VARCHAR(64) NOT NULL UNIQUE,  -- sha256(model|voice|지시문버전|text)
  input_text   TEXT NOT NULL,
  model        VARCHAR(50),                   -- gpt-4o-mini-tts
  voice        VARCHAR(30),                   -- ash
  content_type VARCHAR(50),                   -- audio/mpeg
  audio        BYTEA NOT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

표 3-11. tts_cache 테이블

3.3.7 전체 ERD 요약

테이블	PK	주요 FK / 제약	비고
users	id (BIGSERIAL)	UNIQUE(oauth_provider, oauth_id)	소셜 로그인 전용

테이블	PK	주요 FK / 제약	비고
user_stats	id	user_id → users (1:1)	스트릭·별·학습 시간
user_progress	id	user_id → users UNIQUE(user_id, lesson_id)	레슨 진행·점수·별·이어하기 단계
lessons	id	UNIQUE(lesson_id)	JSONB content, GIN 인덱스, 15개 레슨
word_categories	id	-	덱 14개 (레벨·순서)
words	id	category_id → word_categories UNIQUE(korean_word, pos)	5,561개 단어
fsrs_progress	id	word_id → words UNIQUE(user_id, word_id)	FSRS 상태·안정성·난이도, next_review_date 인덱스
mission_clearances	id	user_id → users	미션 리포트, 판정(cleared)·난이도
ai_cache	id	UNIQUE(input_text, transformation_type)	Gemini 응답 캐시
tts_cache	id	UNIQUE(cache_key)	TTS 음성 캐시

표 3-12. 전체 테이블 요약

레슨·단어 등 콘텐츠 데이터의 변경은 모두 재실행해도 결과가 같은(멱등) SQL 패치 파일(backend/db/patches/*.sql)로 남긴다. 현재 패치는 레슨 4개(lsn_001~004), 미션 2개(msn_001~002), 실험실 1개(lab_001)이며, 개발 DB에서 검증한 패치를 운영 DB에도 그대로 적용하였다.

3.4 인증 설계 (Google · Apple OAuth + JWT)

MARU는 자체 비밀번호 없이 소셜 로그인(Google · Apple OAuth)을 통해 인증한다. 모바일 앱에서 각 플랫폼의 네이티브 SDK가 사용자 인증 후 서명된 idToken을 발급하고, 서버는 이를 검증하여 내부 JWT를 발급한다. 이후 모든 API 요청은 이 JWT를 Authorization 헤더에 담아 전송하며, Flutter의 Dio Interceptor가 이를 자동으로 주입한다.

```
[Flutter App]
  ① Google/Apple 네이티브 SDK 호출 → 사용자 인증
        ↓ idToken (플랫폼 서명 포함)
[Spring Boot  POST /api/auth/google|apple]
  ② idToken 서명 검증 (Google: GoogleIdTokenVerifier, Apple: JWKS Public Key)
  ③ users 테이블 upsert (신규: INSERT / 기존: last_login 갱신)
```

```
  ④ 내부 JWT Access Token 발급 (subject = oauth_id, 유효기간 24h)
        ↓ JWT
[Flutter SecureStorage]
  ⑤ JWT 저장 → 앱 재시작 시 자동 로그인
        ↓ 모든 API 요청
[Dio Interceptor]
  ⑥ Authorization: Bearer <JWT> 자동 주입
        ↓
[JwtAuthenticationFilter]
  ⑦ JWT 서명·만료 검증 → SecurityContext에 oauth_id 저장
  ⑧ 컨트롤러에서 @AuthenticationPrincipal String oauthId 로 사용자 식별
```

그림 3-2. OAuth + JWT 인증 흐름

JWT는 JJWT(io.jsonwebtoken) 라이브러리로 구현하며, 서명 알고리즘은 HMAC-SHA256이다. 토큰 유효기간은 24시간(86,400,000ms)으로 설정하였다. JwtProvider는 토큰 생성·파싱·검증 책임만 가지는 순수 컴포넌트로 분리되어 있으며, JwtAuthenticationFilter가 Spring Security 필터 체인에서 모든 요청을 가로채 검증한다.

idToken 검증에 실패하면 서버는 401과 영어 안내 문장을 반환하고, 앱은 로그인 화면에 오류를 표시한다. iOS에서는 Google 로그인용 서버 클라이언트 ID를 빌드 시 주입하며, 실제 iPhone에서 Google 로그인이 동작함을 확인하였다. 개발 중에는 여러 개발 세션이 동시에 테스트할 수 있도록 디버그 빌드에서만 동작하는 개발용 토큰(DEV_JWT) 로그인을 사용하였다.

3.5 API 설계

MARU 백엔드는 RESTful 설계 원칙에 따라 총 22개의 API 엔드포인트를 제공한다. 모든 응답은 공통 래퍼로 감싸 일관성을 유지한다.

도메인	메서드	엔드포인트	설명
인증	POST	/api/auth/google	Google idToken 검증 → 내부 JWT 발급 (실패 401)
인증	POST	/api/auth/apple	Apple idToken 검증 → 내부 JWT 발급
사용자	GET	/api/me	현재 인증 사용자 정보 조회
사용자	PUT	/api/me/profile	닉네임 등 프로필 설정
사용자	GET	/api/me/stats	학습 통계(스트릭·별·학습 시간·완료 레슨) 조회

도메인	메서드	엔드포인트	설명
레슨	GET	/api/units/{unitId}/lessons	유닛의 공개 레슨 목록 + JSONB content
레슨 진행	POST	/api/progress/lessons/{lessonId}	진행·완료 저장 (점수 기반 별, 최고 기록 유지)
레슨 진행	GET	/api/progress/lessons?unitId=	내 레슨별 진행 기록 조회
음성	GET	/api/tts?text=	한국어 mp3 음성, DB 캐시
단어장	GET	/api/v1/vocabulary/decks	덱 목록 + 단어 수
단어장	GET	/api/v1/vocabulary/decks/{deckId}/lessons	덱의 30단어 레슨 목록 + 학습 진행
단어장	GET	/api/v1/vocabulary/due	레슨 단어 카드 + 평가별 다음 간격
단어장	GET	/api/v1/vocabulary/daily-review	FSRS 복습 기한이 된 단어 조회
단어장	GET	/api/v1/vocabulary/game/{deckId}	짝맞추기용 단어 (라운드당 5쌍)
단어장	POST	/api/v1/vocabulary/review	FSRS 평가(Again/Hard/Good/Easy) 제출
미션챗	POST	/api/v1/mission-chat/setup	LLM #1: 페르소나·미션 생성 (난이도 easy/normal/hard)
미션챗	POST	/api/v1/mission-chat/chat	LLM #2: 한 턴 (토끼+거북이 병렬), 3-Zone 판정
미션챗	POST	/api/v1/mission-chat/suggestion	LLM #4: 거북이 표현 힌트
미션챗	POST	/api/v1/mission-chat/clearance	LLM #3: 목표 달성 판정 + 리포트 저장
미션챗	GET	/api/v1/mission-chat/clearances	내 미션 리포트 목록
랩	POST	/api/lab/explore	Gemini: 한 범주로 3가지 문장 변형
랩	POST	/api/lab/combine	Gemini: 여러 변형자 결합 → 한 문장

표 3-13. API 엔드포인트 목록

인증이 필요한 모든 엔드포인트(인증 API 제외)는 JwtAuthenticationFilter를 통과하며, 컨트롤러 메서드는 @AuthenticationPrincipal 어노테이션으로 현재 사용자를 식별한다. 공통 응답 형식인 ApiResponse는 status, message, data 세 필드로 구성되어 클라이언트가 성공·실패를 일관되게 처리할 수 있다.

3.6 오류 처리와 보안

외부 AI API를 쓰는 서비스는 오류·지연·형식이 깨진 응답을 피할 수 없다. MARU는 이런 상황에서 앱이 멈추거나 예외 원문이 화면에 노출되지 않도록, 모든 기능에 같은 오류 규칙을 적용한다.

HTTP	상황	응답 message 예시
400	입력 누락·형식 오류·범위 밖 값	Please enter a sentence in Korean. / rating must be 1 (Again) to 4 (Easy).
401 / 403	로그인 검증 실패 / 토큰 없음	(로그인 화면에 영어 안내 표시)
404	없는 레슨·단어·사용자	Lesson not found: … / Word not found.
502	AI 오류 응답, AI가 JSON 형식을 깨뜨림	The AI gave an unexpected answer. Please try again.
503	AI 서비스 사용 불가(키 없음·쿼터)	The AI service is unavailable right now. Please try again in a moment.
504	AI 응답 시간 초과	The AI took too long to respond. Please try again.

표 3-14. 공통 오류 응답 규칙 ({status, message, data:null})

message는 학습자에게 그대로 보여줄 수 있는 영어 문장으로 작성하였고, 앱은 각 화면에서 이 문장과 함께 재시도(Retry) 버튼을 제공한다. 실패한 AI 결과는 캐시에 저장하지 않으므로 재시도하면 다시 외부 API를 호출한다. 보안 측면에서는 다음 원칙을 따른다.

- 인증 API 를 제외한 모든 엔드포인트는 JWT 인증이 필요하며, 인증 없이 데이터를 바꿀 수 있는 관리·디버그용 API 를 두지 않는다
- JWT 원문이나 API 키를 로그에 남기지 않는다
- Gemini API 키는 URL 쿼리스트링이 아닌 x-goog-api-key 헤더로 전달한다
- 앱의 서버 주소는 빌드 시 --dart-define 으로 지정하고, API 키·토큰·.env 는 저장소에 커밋하지 않는다

4. 핵심 기능 구현

MARU 는 Korean Lesson(형태소 기반 동적 학습), Vocabulary System(FSRS 및 AI 파이프라인 기반 단어장), Mission Chat(LLM 기반 롤플레이 대화), 그리고 Language Lab(로컬 한글 조합기 및 문장 실험실) 의 네 가지 핵심 기능으로 구성된다. 본 장에서는 각 기능의 설계 의도와 구현 방식을 아키텍처·데이터 모델·핵심 알고리즘 수준에서 구체적으로 기술한다. 백엔드는 Spring Boot(Java 17) 기반의 RESTful API 와 PostgreSQL 을, 프론트엔드는 Flutter 단일 코드베이스를 사용하였다.

4.1 Korean Lesson — 형태소 기반 학습

4.1.1 설계 배경: 교착어를 어떻게 가르칠 것인가

한국어는 어근에 조사·어미가 차례로 결합하는 **교착어**다. 예컨대 '그 식당 어제 갔다 왔는데 생각보다 별로였던 것 같았어요' 라는 문장은 형태소 단위로 '그 / 식당 / 어제 / 가 / 았 / 다 / 오 / 았 / 는데 / 생각 / 보다 / 별로 / 이 / 었 / 던 / 것 / 같 / 았 / 어요'**처럼 매우 잘게 쪼개진다. 형태소를 모두 분해하면 초보 학습자에게는 지나치게 어렵고, 반대로 어절 덩어리만 제시하면 조사·어미가 가진 문법 기능을 학습자가 인지하지 못한 채 넘어가게 된다. MARU 는 이 딜레마를 '선택적 형태소 분할(Selective Morphological Segmentation)' 이라는 원칙으로 해결한다.

이를 두 가지 학습 모드로 구현하였다. 토끼모드는 의미 덩어리(어절) 단위로 먼저 문장을 체험하게 하고, 거북이모드는 해당 레슨이 가르치려는 핵심 조사·어미만 선택적으로 분리하여 구조를 인지시킨다. 즉, 같은 문장을 두 번, 서로 다른 깊이로 체험하게 함으로써 직관과 구조 이해를 모두 확보한다.

```
🐰 토끼: 저는 / Sarah입니다 (의미 덩어리로 체험)
🦕 거북이: 저 / 는 / Sarah입니다 (핵심 조사 '는'만 분리)
```

그림 4-1. 동일 문장에 대한 두 모드의 형태소 분할 차이

4.1.2 목표 문법 형태소 분리 알고리즘

MARU의 형태소 분석은 오픈소스 한국어 형태소 분석기 Kiwi를 사용하되, Kiwi의 원시 분석 결과(Raw Token)를 그대로 노출하지 않고 가공하여 사용한다. 핵심은 어절 안에서 목표 문법에 해당하는 조각만 떼어 내고 나머지는 한 덩어리로 유지하는 것이다. 목표 품사가 들어 있는 어절을 형태소 전부로 쪼개면 목표와 무관한 형태소까지 분해되어 학습 초점이 흐려지기 때문이다. 문장부호·기호는 조립 블록에서 제외하고, 서술격 조사와 어미(이에요·예요·입니다)는 원문 표기 그대로 한 조각으로 묶는다. 이 로직은 backend/admin-tools/kiwi-generator의 core_engine.py에 구현되어 있다.

```
# core_engine.py : process_sentence() – 어절별로 목표 조각만 분리
for u in units:                        # 어절 안의 표면형 조각 (문장부호 제외)
    if self._is_target(u, targets):    # CSV Target_POS 에 해당하는 조각
        flush(buffer)                  # 앞부분은 한 덩어리로
```

```
            pieces.append(piece(u))       # 목표 형태소만 별도 블록
            target_pieces.append((piece(u), u.category))
        else:
            buffer.extend(u)              # 목표가 아니면 이어 붙임
    if not target_pieces:                 # 목표 문법이 없는 어절은 통째로
        pieces = [core]
        explanation = "Keep '저는' as one block — it is not this lesson's focus."


    # 🐰 오답: 목표 형태만 이형태로 바꾼 어절 (저는 → 저은, 학생이에요 → 학생예요)
    # 🦕 오답: 이형태 1순위(은↔는, 을↔를, 이에요↔예요) + 같은 범주 2순위(도, 만 …)
```

그림 4-2. 목표 문법 형태소만 분리하는 핵심 로직 (core_engine.py)

오답 보기(Decoy)는 학습자가 실제로 헷갈리는 선택만 내도록 설계하였다. 받침 유무에 따라 갈리는 이형태(은/는, 이/가, 을/를, 이에요/예요)를 1순위로, 같은 문법 범주의 다른 형태(도·만 등)를 2순위로 사용한다. 무작위 글자 치환처럼 비문을 만드는 오답은 쓰지 않는다.
또한 선택지마다 고유 ID를 부여하여, 같은 형태소가 한 문장에 두 번 필요한 경우(예: '는'×2)에도 풀 수 있게 하였다. 오답 시 보여주는 거북이 설명(turtle_explanation)도 한국어 품사 태그 대신 영어 문장("After a vowel use 를, after a consonant use 을.")으로 생성한다. 실제 레슨 데이터의 분해 결과는 다음과 같다.

레슨 (목표 문법)	문장	🐰 토끼 정답	🦕 거북이 정답
u1-l1 (은/는, JX)	제 이름은 유미예요.	제 / 이름은 / 유미예요	제 / 이름 + 은 / 유미예요
lesson2 (이에요/예요)	저는 학생이에요.	저는 / 학생이에요	저는 / 학생 + 이에요
u1-l3 (을/를, JKO)	저는 커피를 좋아해요.	저는 / 커피를 / 좋아해요	저는 / 커피 + 를 / 좋아해요

표 4-1. 레슨별 목표 문법 분해 결과 (목표가 아닌 "저는"·"좋아해요"는 통째로 유지)

분해 여부는 학습자 개인의 학습 이력이 아니라 문장마다 지정한 목표 문법(Target_POS)으로 결정된다. 예를 들어 을/를 레슨에서 "저는"이 분해되지 않는 이유는 학습자가 이미 배웠기 때문이 아니라, 그 레슨의 목표 문법이 아니기 때문이다. 학습자별 습득 기록에 따라 분해 깊이를 바꾸는 기능은 향후 과제로 남겼다(7.3절).

4.1.3 형태소 드래그앤드롭 퀴즈 생성 파이프라인 (kiwi-generator)

새로운 문법을 가르칠 때마다 코드를 수정해야 한다면 콘텐츠 확장성이 떨어진다. MARU는 데이터와 코드를 격리하는 데이터 주도 설계(Data-Driven Design)를 채택하여, 레슨 행이 있는 경우 curriculum.csv에 문장·번역·목표 품사 한 줄을 추가하고 파이프라인을 실행하면 해당 레슨의 조립 문제가 생성된다. 대상 레슨과 출력 위치는 명령행 인자로 지정하고, 결과는 DB에 직접 쓰는 대신 SQL 패치로 출력한다. 핵심 파이프라인은 kiwi-generator 디렉터리 안의 모듈로 구성된다.

모듈	역할
curriculum.csv	기획자 입력 원천. UnitID · LessonID · Sentence · Translation · Target_POS를 한 행으로 정의
batch_merger.py (관제탑)	명령행 인자(--lesson, --csv, --sql-out, --dry-run 등)로 대상 레슨을 받아 core_engine을 호출. 생성된 퀴즈를 기존 레슨 JSONB의 completion 스텝 바로 앞에 안전하게 병합한 뒤 DB UPDATE를 실행하는 파이프라인 오케스트레이터
core_engine.py (퀴즈 빌더)	Kiwi로 문장을 토큰화하여 어절별 경계를 계산. 타깃 POS 포함 어절만 선택적으로 분해(토끼/거북이 정답)하고, decoy_pool에서 오답 보기를 무작위로 생성. 결과는 schemas.py의 Pydantic 객체로 직렬화
schemas.py (규격 검증)	AgglutinativeQuizData · AgglutinativeElement · AgglutinativeOption를 Pydantic 모델로 선언. 파이프라인 모듈 간 오가는 데이터가 프론트엔드/백엔드 규격을 준수하는지 실시간 검증
db_connector.py (DB 어댑터)	psycopg2 커넥션 풀로 PostgreSQL에 연결·해제를 전담. batch_merger가 생성한 UPDATE 쿼리를 트랜잭션으로 실행

표 4-2. kiwi-generator 파이프라인 모듈

```
# 레슨 하나의 조립 문제를 다시 만들고 SQL 패치로 저장 (DB 직접 수정 대신)
$ python batch_merger.py --lesson u1-l3 --csv curriculum.csv \
      --base-json lessons/unit1/u1-l3.json --chunks \
      --sql-out ../../db/patches/lsn_004_add_u1_l3_object_particle.sql


# batch_merger.py : merge_content() — 기존 조립 단계만 교체
steps = [s for s in base['steps'] if step_type(s) != 'agglutinative_quiz']
idx   = completion_index(steps)              # completion 단계 바로 앞
steps = steps[:idx] + quiz_steps + steps[idx:]
# → UPDATE lessons SET content = '...'::jsonb WHERE lesson_id = 'u1-l3';
```

그림 4-3. 조립 문제 생성·SQL 패치 출력 흐름 (batch_merger.py)

이 파이프라인의 가장 중요한 설계 원칙은 '멱등성(Idempotency)을 보장하는 것'이다. 기존 agglutinative_quiz 단계만 제거한 뒤 새로 생성된 문제를 completion 단계 앞에 삽입하므로, 여러 번 실행해도 소개·연습 등 다른 단계에는 영향이 없고 중복 데이터가 쌓이지 않는다. 결과를 DB에 바로 쓰는 대신 SQL 패치 파일로 남기므로, 개발 DB에서 검증한 것과 같은 내용을 운영 DB에 그대로 적용할 수 있다. 현재 이 파이프라인으로 문법 레슨 3개(u1-l1 은/는, lesson2 이에요/예요, u1-l3 을/를)의 조립 문제 7개를 생성하였다. 새 레슨을 만들 때 소개·연습 단계의 JSON은 사람이 작성한다.

4.1.4 Tap-to-Translate: 형태소 의미 주입 파이프라인

드래그앤드롭 퀴즈 파이프라인과 별개로, MARU는 레슨 intro스텝의 예문 문장에 단어별 형태소 분석 힌트 제공(Tap-to-Translate) 기능을 제공한다. 이는 독립된 두 번째 파이프라인인 generate_morphology.py와 inject_morphology.py로 구현되었으며, 기존 레슨 JSON 스키마에 chunks필드를 추가하여 점진적으로 확장(Backwards-compatible Extension)하는 전략을 택한다.

즉, chunks필드가 없는 구버전 레슨 문장은 기존 텍스트 렌더링으로 폴백(Fallback) 처리하여 호환성을 유지하므로 다른 레슨에 영향을 주지 않는다. generate_morphology.py의 MorphologyGenerator는 Kiwi로 문장을 어절(Chunk) 단위로 분석한 뒤, 각 형태소의 POS 태그를 학습자용 영어 의미로 번역한다. 이를 위해 POS_MAP(품사별 일반 의미)과 TOKEN_SPECIFIC_MAP (고빈도 토큰별 세밀한 의미)을 계층적으로 적용한다.

```
POS_MAP = {
    'JX':  'topic marker',          # 보조사 은/는
    'JKS': 'subject marker',        # 주격조사 이/가
    'JKO': 'object marker',         # 목적격조사 을/를
    'EP':  'tense marker',          # 선어말어미 았/었
    'EF':  'polite/formal ending',
}
TOKEN_SPECIFIC_MAP = {              # 토큰별 세밀한 의미 (우선 적용)
    '저': 'I (polite)',   '나': 'I (informal)',
    '입니다': 'am/is/are (formal)',
}
# 적용 우선순위: TOKEN_SPECIFIC_MAP > POS_MAP > 원문 그대로
```

그림 4-4. POS 태그 → 학습자 언어 계층적 매핑 (generate_morphology.py)

inject_morphology.py는 레슨 JSON 파일의 intro스텝을 순회하여 각 문장 객체의 korean필드(한국어 원문)를 MorphologyGenerator에 넘기고, 반환된 MorphologyChunk 리스트를 chunks필드로 주입한다. 주입된 데이터는 schemas.py의 MorphologyChunk 및 MorphologyToken Pydantic 모델로 구조화된다. 최종적으로 Flutter 앱은 문장의 어절을 탭할 때 해당 청크의 토큰 배열을 순회하여 힌트 패널을 렌더링한다.

```
# inject_morphology.py : chunks 주입 핵심 흐름
for sentence in intro_step['content']['sentences']:
    chunks = gen.analyze(sentence['korean'])   # -> List[MorphologyChunk]
    sentence['chunks'] = [c.dict() for c in chunks]  # JSON 직렬화


# 결과 예시 (저는 Sarah입니다.)
# chunks: [
#   { display: '저는', tokens: [{text:'저', meaning:'I (polite)'},
#                               {text:'는', meaning:'topic marker'}] },
#   { display: 'Sarah입니다.', tokens: [{text:'Sarah입니다.', meaning:'am/is/are
(formal)'}] }
# ]
```

그림 4-5. 레슨 문장에 형태소 청크 주입 (inject_morphology.py)

탭 분석 데이터에서 기호(":", "=" 등)는 탭 대상에서 제외하고, 이름은 "Sarah (name)", 화자 표시는 "Minsu (speaker)"처럼 일관되게 표시하며, 서술격은 원문 표기(예요/이에요/입니다)를 따른다. 뜻풀이가 정적 규칙표에 의존한다는 한계는 7.2.1절에서 다룬다.

4.1.5 백엔드 주도(Backend-Driven) 동적 렌더링과 JSONB 설계

레슨의 UI는 클라이언트에 하드코딩되지 않는다. 서버가 내려준 레슨 데이터의 구조에 따라 Flutter가 위젯을 동적으로 조립 및 렌더링 한다. 데이터 흐름은 다음과 같다.

```
PostgreSQL (JSONB)
      │
      ▼  GET /api/units/{unitId}/lessons
Spring Boot API  (LessonController · LessonService)
      │
      ▼  LessonModel.fromJson()
Flutter Model Parsing
      │
      ▼  StepRenderer (stepType별 위젯 분기)
Dynamic Widget Rendering
```

그림 4-6. Backend-Driven Lesson System 데이터 흐름

레슨마다 세부 스텝(Step)의 구조가 완전히 다르다는 점이 설계의 핵심 난제였다. 예컨대 intro스텝은 설명용 텍스트 및 오디오, agglutinative_quiz스텝은 정답 및 오답 보기 배열, completion스텝은 학습 완료 데이터 구조를 가진다. 이를 정규화된 RDB 테이블로 설계하면 새로운 스텝 유형(stepType)이 추가될 때마다 신규 테이블 생성 및 DB 마이그레이션이 필요하다. MARU는 대신 PostgreSQL의 JSONB 비정형 컬럼을 선택하였다.

구분	정규화 RDB 설계	JSONB 설계 (채택)
스텝 추가 시	테이블 신설 + 마이그레이션 필요	스키마 변경 없이 즉시 수용
저장 구조	유형별 테이블 분산	이종(異種) 구조를 단일 컬럼에 저장
검색	표준 인덱스	GIN 인덱스로 JSON 내부 검색 가능
특성	정합성은 높으나 경직	RDB 안정성 + NoSQL 유연성 동시 확보

표 4-3. 정규화 설계와 JSONB 설계 비교

구현상으로는 엔티티에서 Hibernate 6의 @JdbcTypeCode(SqlTypes.JSON)을 사용하여 DTO 객체와 JSONB 컬럼 간 직렬화를 자동화하였다. 따라서 서비스 계층은 별도 변환 코드 없이 객체를 그대로 반환할 수 있다.

```
@JdbcTypeCode(SqlTypes.JSON)
@Column(columnDefinition = "jsonb", nullable = false)
private LessonContentDto content;  // 객체 <-> JSONB 자동 직렬화
```

그림 4-7. JSONB 자동 매핑 (Lesson.java)

4.1.6 점수·별·이어하기

레슨 점수는 문제별 첫 시도 정답률이다. 앱이 점수를 전송하면 서버가 점수에 따라 별을 계산한다

(80점 이상 3개, 60점 이상 2개, 그 외 1개). 다시 풀어도 최고 기록이 유지되며, 늘어난 별만 사용자 통계에 더해진다. 레슨 도중 나가면 현재 단계가 저장되어 다음에 이어서 학습할 수 있고, 레슨 목록에는 완료 표시와 별이 나타난다(GET /api/progress/lessons).

한글 레슨(Unit 0, 12개)은 자모·음절 80항목을 소개 카드 → 따라 읽기 → 듣기 퀴즈 순서로 학습한다. 카드 설명과 완료 화면 문구는 모두 레슨 데이터에서 오며, 듣기 퀴즈는 항목 수만큼 중복 없이 출제하고 실제 점수를 표시한다.

4.1.7 서버 TTS — 자연스러운 한국어 음성

기기 내장 TTS는 기기마다 음질이 다르고 학습용으로 듣기에 부자연스러운 경우가 많다. MARU는 서버가 OpenAI gpt-4o-mini-tts(목소리 ash, "표준 서울말, 학습자용으로 약간 천천히" 지시)로 mp3를 생성하고 tts_cache 테이블에 저장한다. 같은 문장은 OpenAI를 다시 호출하지 않고 캐시에서 바로 반환하며, 응답 헤더 X-TTS-Cache로 HIT/MISS를 확인할 수 있다.

```
GET /api/tts?text=저는 커피를 좋아해요   (인증 필요, 1~200자)
 ├ text 정규화(앞뒤·연속 공백) → 자모 한 글자는 표준 읽기로 변환 (ㄱ → 기역, ㅏ → 아)
 ├ cache_key = sha256(model | voice | 지시문버전 | text)
 ├ tts_cache HIT  → audio/mpeg 즉시 반환 (수 ms)
 └ tts_cache MISS → OpenAI TTS 호출(약 1초) → 저장 → 반환
앱: TtsHelper.speak(text) 한 곳에서 just_audio 재생
     실패(4xx/5xx·타임아웃) 시 기기 TTS(flutter_tts)로 폴백
```

그림 4-8. 서버 TTS 처리 흐름

앱은 레슨·단어장·실험실이 모두 같은 TtsHelper.speak()를 호출하며, 서버 음성을 받지 못하면 기기 TTS로 대체한다.

4.2 Vocabulary System — 적응형 단어 학습

Vocabulary System은 망각곡선 모델 기반의 간격 반복 학습(Spaced Repetition)을 통해 학습자의 단어 암기 효율을 극대화하는 것을 목표로 한다. 단어 데이터 구축→ 덱(deck) 구성 → 최초 학습 → 일일 복습(Daily Review) → 매칭 게임으로 이어지는 학습 사이클을 제공하며, 그 중심에는 망각곡선 기반 간격 반복 알고리즘이 있다.

4.2.1 AI 데이터 파이프라인 (단어 데이터 자동 구축)

단어 데이터는 국립국어원이 배포한 학습용 어휘 XLS(5,965개)를 원천으로 한다. 이를 사람이 일일이 번역·분류하는 대신, Gemini를 활용한 3단계 자동 파이프라인(vocab-pipeline)으로 구축하였다. 최종 적재된 단어는 14개 주제 덱의 5,561개이며, 번역·예문은 AI가 생성한 것이다.

```
국립국어원 XLS (5,965개)
   │  [Step 1] parse_vocab_xls.py  – 정규화
   ▼
VocabEntry 목록
   │  [Step 2] gemini_translator.py + Gemini API
   ▼              (번역 / 예문 / 카테고리 분류)
TranslationResult 목록
   │  [Step 3] db_inserter.py  – PostgreSQL Bulk Insert
   ▼
vocabulary 테이블
```

그림 4-9. AI 단어 데이터 파이프라인 3단계

이 파이프라인은 "기초적인 한국어 표제어와 한자 힌트가 포함된 단어 목록만 있으면 예문 및 다국어 번역이 결합된 학습용 단어 데이터를 자동으로 구축할 수 있는 구조"로 설계되었으며, 안정성과 비용을 위해 두 가지 핵심 설계를 적용하였다.

4.2.2 안정성과 비용을 위한 두 가지 설계

단어 하나당 한 번씩 호출하면 약 6,000회의 API 호출이 발생한다. 15개(batch_size) 단위로 단어를 묶어 한 번의 프롬프트로 처리한다. 그 결과 호출 횟수가 약 400회로 감소하여 비용을 약 1/15로 절감하였다.

```
def __init__(self, api_key=None, batch_size=15, max_retries=3):
    self.batch_size = batch_size


def translate_all(self, entries):
    # 15개 단위로 분할하여 순차 배치 처리
    batches = [entries[i:i+self.batch_size]
               for i in range(0, len(entries), self.batch_size)]
```

그림 4-10. 15개 단위 배치 처리 (gemini_translator.py)

파이프라인이 중간에 실패하여 재실행되더라도 중복 데이터가 쌓이면 안 된다.
words테이블은 복합키에 대해 ON CONFLICT (korean_word, part_of_speech) DO NOTHING제약 조건을 적용하여, 재실행 시에도 중복 적재 0건을 보장한다.

```
INSERT INTO words (korean_word, part_of_speech, ...) VALUES (...)
ON CONFLICT (korean_word, part_of_speech) DO NOTHING;
-- 파이프라인 재실행해도 중복 0건 (멱등성 보장)
```

그림 4-11. 멱등성 보장 INSERT (db_inserter.py)

또한 한국어 동음이의어 문제를 한자 메타데이터 주입으로 해결하였다. 예컨대 "가구"만 던지면 AI가 가구(家具: 가재도구)인지 가구(家口: 세대주와 세대원)인지 구분하지 못한다. 원천 데이터의 한자 메

타데이터를 (한자: 한자 힌트) 형태로 프롬프트에 주입함으로써 번역 정확도를 확보하였다.

```
def build_batch_prompt(entries: list[VocabEntry]) -> str:
    """
    VocabEntry 배치를 Gemini에 전달할 프롬프트로 변환.
    한자 풀이(hanja_hint)가 있으면 번역 정확도 향상을 위해 힌트로 포함.
    """
    word_lines = []
    for e in entries:
        hint = f" (한자: {e.hanja_hint})" if e.hanja_hint else ""
        word_lines.append(f"- {e.korean_word} [{e.part_of_speech}]{hint}")

    words_block = "\n".join(word_lines)

    return f"""아래 한국어 단어 목록을 외국인 초보 학습자(A1~B2 수준)를 위해 번역하고 분류해 주세요.

단어 목록:
{words_block}

각 단어에 대해 다음 JSON 배열 형식으로만 응답해 주세요. 다른 설명은 절대 추가하지 마세요:
```json
[
  {{
    "korean": "단어",
    "english": "가장 일반적인 영어 번역 (동사는 'to + 동사원형' 형식)",
    "category": "의미 기반 카테고리 (예: 음식, 동물, 감정, 장소, 동작, 신체, 교통, 날씨, 가족, 직업, 학교, 쇼핑, 시간, 숫자,
색상, 기타)",
    "example_kr": "10단어 이내의 자연스러운 한국어 예문",
    "example_en": "위 예문의 영어 번역"
  }}
]
```"""
```

그림 4-12. 배치 번역 프롬프트 (gemini_translator.py)

4.2.3 FSRS 기반 간격 반복(Spaced Repetition) 알고리즘

복습 스케줄링의 핵심은 FSRS(Free Spaced Repetition Scheduler) 알고리즘이다. FsrsAlgorithm은 FSRS의 상태 모델(New·Learning·Review·Relearning)과 공개된 FSRS-4.5 표준 공식, 기본 파라미터 17개(w0~w16)를 그대로 구현한다. 각 단어 카드에 대해 안정성(S, 기억이 유지되는 기간), 난이도(D, 1~10), 인출 가능성(R, 지금 기억하고 있을 확률)을 계산하고, R이 목표 기억률 0.9로 떨어지는 시점을 다음 복습일로 정한다. 구현체는 DB·JPA에 의존하지 않는 순수 자바 객체로 분리하여 단위 테스트로 검증하였다.

사용자의 4단계 응답(G: Again=1, Hard=2, Good=3, Easy=4)에 따라 새 카드는 초기 안정성 S0 = w[G−1]과 초기 난이도 D0 = w4 − (G−3)·w5로 시작한다. 복습 카드는 경과일 t로 R(t,S)를 구한 뒤, 기억에 성공하면 R이 낮을수록(잊기 직전에 떠올릴수록) 안정성이 크게 늘고, 실패(Again)하면 망각 공식으로 안정성이 줄며 5분 뒤 다시 출제된다.

```
// FsrsAlgorithm.java — FSRS-4.5 표준 공식 (기본 파라미터 w0~w16)
R(t,S)   = (1 + FACTOR·t/S)^DECAY             // DECAY = -0.5, FACTOR = 19/81
I(S)     = S/FACTOR · (0.9^(1/DECAY) − 1)      // 목표 기억률 0.9 → 간격 = S일
D'       = w7·D0(Good) + (1−w7)·(D − w6·(G−3)) // 평균 회귀, 1~10
S'recall = S·(1 + e^w8·(11−D)·S^−w9·(e^(w10·(1−R)) − 1)·hardPenalty·easyBonus)
S'forget = w11·D^−w12·((S+1)^w13 − 1)·e^(w14·(1−R))


double nextRecallStability(double d, double s, double r, ReviewRating g) {
    double hardPenalty = g == ReviewRating.HARD ? W[15] : 1;
    double easyBonus   = g == ReviewRating.EASY ? W[16] : 1;
    return s * (1 + Math.exp(W[8]) * (11 - d) * Math.pow(s, -W[9])
            * (Math.exp((1 - r) * W[10]) - 1) * hardPenalty * easyBonus);
}
```

그림 4-13. FSRS-4.5 핵심 공식과 구현 (FsrsAlgorithm.java)

응답 등급	새 카드 (S, D)	다음 복습	복습 카드에서
Again	S=0.49, D=7.62	5분 뒤 (Learning)	Relearning, 망각 공식으로 S 감소, 5분 뒤
Hard	S=1.40, D=6.39	1일 뒤	S 소폭 증가 (hardPenalty)
Good	S=3.71, D=5.16	4일 뒤	예: 4일 뒤 Good → S 3.71 → 14.81, 15일 뒤
Easy	S=13.82, D=3.93	14일 뒤	S 크게 증가 (easyBonus), 간격 Hard ≤ Good < Easy 보장

표 4-4. FSRS-4.5 기본 파라미터에서의 평가별 결과 (서버 실측·손계산 검증)

계산 결과는 손계산으로 검증하였으며, 공식 구현체(py-fsrs)와의 교차 검증과 사용자 데이터로 파라미터를 최적화하는 기능은 아직 적용하지 않았다(7.2.5절).

4.2.4 일일 복습(Daily Word Review)과 매칭 게임

FSRS가 계산한 복습 기한이 된 단어만을 선별하여 "오늘의 복습(Daily Review)"으로 제공한다. 요청 시 FsrsProgressRepository가 @Query(JPQL) 조건으로 복습 도래 카드를 조회하며, state 정렬로 재학습 카드를 우선 노출한다. 복습 결과는 다시 4단계 평가를 거쳐 FSRS로 재연산된다. 홈 화면에는 복습할 단어 수가 항상 표시되며, 0개이면 "All caught up!" 카드로 안내한다. 덱 안의 Word Study는 레슨(30단어)의 단어를 순서대로 학습하는 모드로, FSRS 추천은 오늘의 복습에 한정된다.

```
@Query("SELECT f FROM FsrsProgress f JOIN FETCH f.word w " +
       "WHERE f.userId = :userId AND f.nextReviewDate <= :now " +
       "ORDER BY f.state ASC, f.nextReviewDate ASC")
List<FsrsProgress> findDueCardsByUser(userId, now, pageable);
```

그림 4-14. 복습 도래 단어 조회 쿼리 (FsrsProgressRepository.java)

암기를 게임으로 강화하기 위해 단어 짝맞추기 게임(Match Madness)을 제공한다. 백엔드가 레슨(30단어)의 단어를 내려주고, 앱은 라운드당 5쌍(한글 ↔ 영어 뜻 2열)씩 나누어 출제한다. 라운드 수와 진행 표시는 실제 단어 수로 계산한다. 타일 순차 등장, 선택 시 확대, 정답 짝 팝·페이드, 오답 흔들림, 라운드 전환, 완료 축하 연출이 있으며, 토끼 캐릭터가 정답·오답에 반응한다.

4.2.5 평가 무결성과 다음 간격 미리보기

FSRS는 평가 기록이 정확해야 의미가 있다. 카드를 스와이프로 넘겨도 화면에 보이는 카드와 평가가

저장되는 단어가 어긋나지 않도록 카드 인덱스와 상태를 동기화하고, 평가는 항상 해당 카드의 단어 ID로 제출한다. 서버는 범위 밖 평가(1~4 외)나 없는 단어를 400/404로 거절한다.

Word Study에서 이미 학습한 단어를 복습일 전에 다시 평가하면 간격이 흐트러지므로, 이 경우는 저장하지 않는다(Spacing Integrity Guard, 응답 applied:false). 단, 복습일이 지난 단어의 평가는 정식 복습으로 반영한다. 또한 서버가 네 평가 각각의 결과를 미리 계산(nextIntervals)해 내려주어, 평가 버튼에 "Again 5m · Hard 1d · Good 4d · Easy 14d"처럼 다음 복습 간격을 표시한다. 학습자는 자신의 선택이 복습 일정에 어떤 영향을 주는지 바로 알 수 있다.

4.3 Mission Chat — 역할극 미션 대화

범용 LLM으로 외국어 대화를 연습할 때 학습자는 세 가지 문제에 직면한다. 목적 부재(상황·목적·상대가 없음), 끝없는 대화(종료 기준 부재), 피드백 부재이다. MARU는 "LLM은 훌륭한 엔진이지만, 진짜 학습이 일어나는 교육학적 궤도는 사람이 설계해야 한다"는 전제 아래, 이 세 문제를 각각 사용자 페르소나 및 미션 생성, 3-Zone 기반의 턴 제어, 그리고 거북이 코치의 실시간/최종 피드백 제공으로 해소한다.

4.3.1 4 단계 LLM 호출 구조

Mission Chat은 역할이 분리된 네 번의 LLM 호출로 구성된다. 서버는 대화 상태를 저장하지 않고(Stateless), 클라이언트가 매 턴 전체 대화 이력과 설정을 전송하는 방식으로 수평 확장성을 확보하였다.

호출	엔드포인트	역할
#1 Setup	POST /setup	사용자 설정(위계·친밀도·역할)으로 페르소나·미션 생성
#2 Chat	POST /chat	한 턴 처리 — 토끼(연기) + 거북이(채점) 병렬 호출
#3 Clearance	POST /clearance	미션 종료 시 목표 달성 판정 + 피드백 리포트 저장
#4 Suggestion	POST /suggestion	막힌 학습자에게 거북이가 표현 힌트 제공

표 4-5. Mission Chat의 LLM 호출 구조

4.3.2 비동기 병렬 대화 처리 (토끼·거북이 역할 분리)

한 번의 대화 턴에서 MARU는 두 개의 서로 다른 페르소나를 동시에 가동한다. 토끼(Rabbit)는 페르소나에 몰입하여 자연스러운 대화 응답을 하고, 거북이(Turtle)는 학습자의 발화를 문법적으로 평가 및 교정한다. 각 LLM이 하나의 목표에만 집중하도록 시스템 프롬프트(rabbit_reply_system.txt, turtle_eval_system.txt)를 완전히 분리하였다.

두 호출을 순차로 처리하면 응답 지연이 누적된다. ChatTurnService는 Java의 CompletableFuture를 사용해 두 호출을 비동기 병렬로 실행하고, CompletableFuture.allOf().join()으로 안전하게 병합하여 단일 DTO로 반환한다. 서버 로그로 측정한 결과 한 턴의 응답은 약 1.5~2초(실측 1.4~2.2초)로, 두 호출을 순차 처리할 때의 추정치(2.6~4.1초)보다 30~50% 짧았다.

```
// ChatTurnService.java : processTurn()
CompletableFuture<String> rabbitFuture = CompletableFuture
    .supplyAsync(() -> openAiService.askWithHistory(rabbitPrompt, history));
CompletableFuture<String> turtleFuture = CompletableFuture
    .supplyAsync(() -> openAiService.askWithHistory(turtlePrompt, history));


// 두 비동기 호출을 병렬 실행 후 안전 병합
CompletableFuture.allOf(rabbitFuture, turtleFuture).join();
return parseResponses(rabbitFuture.get(), turtleFuture.get());
```

그림 4-15. 토끼·거북이 병렬 비동기 호출 (ChatTurnService.java)

4.3.3 3-Zone 턴 제어 시스템

"대화에 끝이 없다"는 문제를 해결하기 위해, MARU는 자연어 기반 LLM에 대화 상태 제어 규칙을 적용하였다. 현재까지의 사용자 발화 횟수(current_turn_count)와 미션의 기준 턴 수(mission_min_turns, 목표 대화 길이)를 시스템 프롬프트에 주입하여, LLM이 대화의 진행 단계를 3-Zone(Zone A: Early stage, Zone B: Judgment window, Zone C: Overtime)으로 인지하고 미션 클리어 조건 충족 여부(mission_status)를 스스로 판정하도록 제어한다. 다만 LLM의 판단만으로는 규칙이 항상 지켜진다고 보장할 수 없으므로, 서버가 LLM의 판정 위에 3-Zone 규칙을 강제한다(표 4-6). 그 결과 모든 미션은 최대 minTurns+3 턴 안에 반드시 끝난다.

Zone	조건 (t = 사용자 턴)	서버 규칙
A (초반)	t < minTurns − 1	LLM이 cleared/failed를 주어도 무시하고 진행 (in_progress)
B (판정 구간)	minTurns − 1 ≤ t ≤ minTurns + 1	목표 달성이면 cleared, 아니면 계속 진행
C (연장)	t ≥ minTurns + 2	cleared 또는 failed. t ≥ maxTurns(= minTurns + 3)인데 진행 중이면 서버가 failed로 종료

표 4-6. 서버가 강제하는 3-Zone 규칙 (ChatTurnService.applyZoneRules)

미션이 cleared 또는 failed로 끝나면 앱은 입력을 막고 호출 #3을 요청한다. 목표를 달성하지 못한 채 끝난 경우에도 호출하며, 거북이 코치가 목표 달성 여부를 판정하고 피드백 리포트를 만든다(4.3.5절). 막연한 AI 잡담이 아니라, 정해진 턴 안에 끝나는 목적 지향형 학습 대화로 완성되는 것이다.

4.3.4 사용 프롬프트

Mission Chat은 역할별로 분리된 다섯 개의 시스템 프롬프트를 사용한다. 프롬프트는 서버 리소스(prompts/*.txt)로 관리하며, {{변수}} 자리에 미션 설정·난이도 규칙·대화 상태를 채워 호출한다. 모든 프롬프트는 마크다운 없이 정해진 JSON만 출력하도록 지시하고, 서버는 필수 필드가 없거나 형식이 깨지면 오류(502)로 처리한다.

프롬프트 파일	역할	주요 규칙	출력(JSON)
mission_setup_system.txt	사용자 설정(관계·친밀도·역할·성격·난이도)으로 토끼의 페르소나와 미션 생성	미션은 사용자 관점으로 작성. 어색한 설정 조합은 자동 조정하고 이유를 안내(adjustment_notice). 목표 대화 길이(기준 턴)는 난이도별 범위 안에서 지정	persona, mission(title, description, clear_condition, min_turns), adjustment_notice
rabbit_reply_system.txt	토끼: 페르소나에 몰입해 대화 응답	학습자의 실수를 직접 교정하지 않음. 난이도별 답 길이 상한. 3-Zone 기준으로 mission_status 판단. cleared 시 자연스러운 마무리	rabbit_reply, rabbit_reply_en, mission_status
turtle_eval_system.txt	거북이: 학습자의 마지막 발화를 평가·교정	관계 맥락을 고려(상사가 부하에게 반말은 오류 아님). 심각도 immediate / side / none 구분, 난이도별 교정 강도	severity, issue_type, correct_expression, turtle_feedback(+en)
turtle_suggestion_system.txt	거북이: 막힌 학습자에게 힌트 문장 제안	격식·관계·친밀도에 맞는 2~3개 문장, 미션 목표로 이어지는 실용적 표현, 난이도별 문장 길이	suggestions[korean, english]
clearance_system.txt	거북이 코치: 목표 달성 판정과 피드백 리포트	후보로 주어진 학습자 문장만 원문 그대로 인용. 잘한 표현 0~4개, 고칠 표현 0~3개. 판정 이유는 학습자에게 말하는 2인칭	result, result_reason, good_expressions, incorrect_expressions, turtle_comment, next_practice

표 4-7. Mission Chat 시스템 프롬프트 구성

```
=== rabbit_reply_system.txt : OUTPUT FORMAT (strict JSON, no markdown) ===
{
  "rabbit_reply": "페르소나의 한국어 응답",
  "rabbit_reply_en": "English translation of rabbit_reply",
  "mission_status": "in_progress | cleared | failed"
}
=== turtle_eval_system.txt : OUTPUT FORMAT ===
```

```
{
  "severity": "immediate | side | none",
  "issue_type": "honorific_mismatch | grammar_error | vocabulary | pragmatic | off_topic |
none",
  "user_input_problematic": "문제가 된 표현 (없으면 null)",
  "correct_expression": "올바른 표현 (없으면 null)",
  "turtle_feedback": "거북이 피드백 멘트 (없으면 null)",
  "turtle_feedback_en": "English translation of turtle_feedback"
}
=== clearance_system.txt : OUTPUT FORMAT ===
{ "certificate": {
    "result": "cleared | not_cleared",
    "result_reason": "One sentence in English addressed to the learner, starting with \"You\"",
    "good_expressions":      [ { "expression": "한국어 표현", "reason": "Explanation in
English" } ],
    "incorrect_expressions": [ { "wrong": "틀린 표현", "correct": "올바른 표현", "explanation":
"..." } ],
    "turtle_comment": "One line summary in English",
    "next_practice": "Next goal in English" } }
```

그림 4-16. 토끼·거북이·리포트 프롬프트의 출력 형식 (발췌)

4.3.5 목표 달성 판정과 피드백 리포트

미션이 끝나면 거북이 코치(호출 #3)가 대화 전체를 보고 미션 목표(goalCondition)와 언어 조건 달성 여부를 판정한다. 달성하면 "Mission Cleared", 미달성이면 "Almost there!"와 함께 판정 이유(resultReason)와 재도전 조언을 보여준다. 어떤 경우든 잘한 표현·고칠 표현·다음 목표 피드백은 항상 제공한다. 대화가 minTurns−1 턴보다 짧게 끝나면 서버가 AI 판정과 무관하게 미달성으로 처리한다.

리포트에 인용되는 문장은 학습자가 실제로 보낸 문장이어야 한다. LLM은 힌트로 제안만 된 문장처럼 학습자가 말하지 않은 문장을 인용할 수 있으므로, 서버가 대화 기록에서 학습자 발화만 뽑아 번호 목록으로 프롬프트에 넣고, AI 응답의 각 인용 문장이 실제 학습자 문장에 포함되는지 정규화(유니코드 NFC·공백·문장부호) 후 검사하여 아니면 제거한다.

```
// MissionClearanceService.java — AI 가 인용한 문장이 실제 사용자 발화인지 검증
List<String> userSentences = userSentences(conversationHistory);  // role=user 만
promptVars.put("student_messages", numberedList(userSentences));  // 후보로 제공
...
if (!isQuotedFromUser(item.path("expression").asText(""), userSentences)) {
    continue;                                    // 후보 밖 인용은 버림
}
static boolean isQuotedFromUser(String quote, List<String> userSentences) {
    String q = normalizeForMatch(quote);         // NFC · 공백 · 문장부호 정규화
    return userSentences.stream().anyMatch(u -> normalizeForMatch(u).contains(q));
}
```

그림 4-17. 리포트 인용 문장 검증

4.3.6 레드카드·옐로카드와 토끼 변신 로딩

거북이의 실시간 교정은 심각도에 따라 두 가지 카드로 표시한다. 존댓말 수준이 틀리거나 대화가 성

립하지 않는 치명적 오류(immediate)는 레드카드로 표시하고 해당 문장을 전송하지 않으며("Not sent"), 대화는 이어갈 수 있는 사소한 실수(side)는 옐로카드로 표시한다. 카드에는 "Politeness level", "Grammar" 같은 사람이 읽는 라벨을 붙였다. 미션 설명 패널은 기본으로 접혀 제목·턴 수(Turn n/max)·목표 한 줄만 보이고, 탭하면 펼쳐진다.

미션 생성에는 수 초가 걸리므로, 로딩 화면을 "장난꾸러기 토끼가 사용자가 고른 역할로 변신하는 중"이라는 컨셉으로 만들었다. 사용자가 입력한 역할이 문구에 들어가고(예: Tokki is transforming into a café barista...), 마술봉을 든 토끼(magic 표정)가 회전·반짝임 모션으로 변신한다. GPT가 낱자 모음을 조합형 자모로 출력하면 iOS에서 글자가 깨지므로(□), 앱은 이를 표준 자모로 변환해 표시한다.

4.3.7 난이도 설계 — Easy·Normal·Hard

하나의 프롬프트로 모든 학습자를 상대하면 초급 학습자는 토끼의 긴 문장을 읽느라 대화를 이어가기 어렵다. MARU는 미션 설정 화면에서 난이도를 Easy·Normal·Hard 중에서 고르게 하고(기본값 Easy), 선택한 난이도를 프롬프트와 서버 규칙 양쪽에 적용한다. 난이도는 /setup 요청의 선택 필드(difficulty)로 전달되며, 값이 없거나 알 수 없으면 easy로 처리한다.

항목	Easy (기본)	Normal	Hard
어휘·문법	TOPIK 1, 현재형-·요 위주	TOPIK 2, 기본 연결어미	자연스러운 구어
미션 목표	1개, 단순	1~2단계	여러 단계
목표 대화 길이(기준 턴)	3~4턴 안팎	4~6턴 안팎	5~10턴 안팎
토끼 답 길이 상한	1문장, 공백 빼고 약 25자	1~2문장, 약 50자	최대 2문장, 약 80자
거북이 교정 강도	뜻이 안 통하는 큰 실수만. 존댓말 실수도 옐로카드(막지 않음)	분명한 문법·어휘·존댓말	화용(드시다·계시다 등)까지
힌트 문장	아주 짧고 쉬운 -요 문장	짧은 문장	자연스러운 문장
수료 판정	목표만 달성하면 통과(실수에 관대), 고칠 표현 2개 이하	보통	엄격

표 4-8. 미션 난이도별 서버 규칙

토끼 답이 상한의 1.4배를 넘으면 서버가 토끼 호출을 한 번 더 요청하여 더 짧은 답을 받는다. 답을

잘라내지는 않으므로 문장이 어색해지지 않는 대신, 그 턴만 응답이 약 1초 늦어질 수 있다. 목표 대화 길이는 반드시 채워야 하는 턴 수가 아니라 대화 분량의 기준이다. 목표를 이루면 기준 턴 무렵에 마무리되고, 기준 +3턴 안에 반드시 종료된다(3-Zone 규칙은 모든 난이도에서 같다). 리포트(수료증)와 목록에는 그 미션의 난이도 배지가 표시되고, 난이도는 mission_clearances.difficulty 컬럼에 저장된다. 다만 난이도는 학습자가 직접 고르는 방식이며, 학습자의 실력을 자동으로 측정해 조절하는 기능은 아니다(7.2.2절).

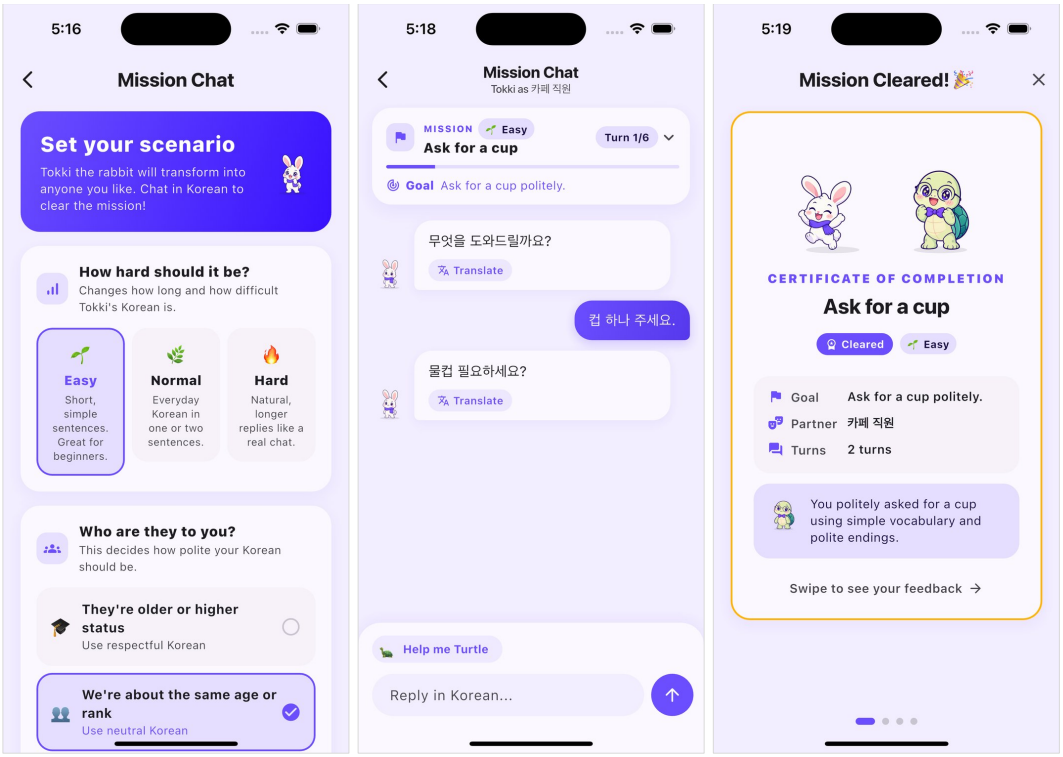


그림 4-18. 난이도 선택(Easy·Normal·Hard), Easy 미션의 짧은 토끼 답, Easy 배지가 붙은 수료증

4.4 Language Lab — 실험형 인터랙티브 학습

Language Lab은 학습자가 직접 한국어를 "실험"하며 표현을 확장하는 인터랙티브 기능으로, Hangeul Lab(로컬 한글 조합기)과 Sentence Lab(문장 실험실) 두 트랙을 제공한다. 전자는 외부 API 없이 로컬 연산만으로, 후자는 AI와 캐싱 아키텍처를 결합하여 구현하였다 — 대조적인 두 설계가 한 기능 안에 공존한다.

4.4.1 Hangeul Lab — 유니코드 한글 조합

학습자가 초성·중성·종성 버튼을 탭하면 즉시 완성된 한글 글자가 생성되고 TTS로 발음을 들려준다. 핵심은 유니코드 한글 조합 공식만으로 로컬에서 즉시 글자를 만드는 것이다. 한글 음절은 유니코드에서 초성, 중성, 종성의 순서를 기준으로 규칙적으로 배열되어 있어, 다음 수식으로 계산된다.

```
// hangeul_combiner.dart
static String combine(int cho, int jung, int jong) {
  // 유니코드 한글 조합 공식 (Base: 0xAC00 = '가')
  int unicode = 0xAC00
      + (cho  * 21 * 28)   // 초성 (19종)
      + (jung * 28)        // 중성 (21종)
      + (jong > 0 ? jong : 0); // 종성 (28종, 선택)
  return String.fromCharCode(unicode);
}
// ㄱ(0) + ㅏ(0) = 0xAC00 = '가' — 즉시 생성
```

그림 4-19. 유니코드 한글 조합 공식 (hangeul_combiner.dart)

상태 관리는 Flutter의 Riverpod 상태 관리를 사용하여 처리하며, 흐름이 외부 통신 없이 단말에서 완결된다. 네트워크 지연이 전혀 없어 한글 창제 원리를 손끝으로 즉각 체험할 수 있다.

4.4.2 Sentence Lab — AI 기반 문장 변형 실험

Sentence Lab은 학습자가 입력한 문장을 시제·존댓말·부정 등 원하는 문법 범주로 직접 변형해 보는 기능이다. 두 가지 모드를 제공한다. Explore 모드는 한 문법 범주에 대해 3가지 변형 문장과 영어 설명을 생성하고, Combine 모드는 여러 변형자(예: 과거+반말)를 결합한다 — 교착어의 다중 형태소 결합 특성을 직접 실험하게 한다. 두 모드 모두 Gemini API를 호출하되, 결과를 순수 JSON 포맷으로 강제하는 프롬프트 설계를 적용하였다.

```
// AiLabService.java : buildExplorePrompt()
String prompt = """
  You are the world's best Korean grammar expert.
  Modify the following Korean sentence into 3 different
  versions based on the given [Category].
  Respond ONLY with a pure JSON Array.
  Input Sentence: "%s"   Category: "%s"
  [{ "text": "...", "type": "Honorific",
     "explanation": "..." }]
  """.formatted(inputText, category);
```

그림 4-20. 문법 변형 프롬프트 (AiLabService.java)

4.4.3 AI 캐싱 아키텍처와 정렬 기반 캐시 키

AI 호출은 비용과 지연을 수반하므로, MARU는 동일 요청에 대해 ai_cache 테이블을 활용한 캐싱 레이어를 둔다. 요청이 들어오면 입력 문장과 변형 유형으로 캐시를 먼저 조회하여, 캐시 적중(Cache Hit) 시 1~2ms 만에 반환하고, 캐시 미스(Cache Miss) 시에만 Gemini를 호출(3~5초)한 뒤 결과를 검증·저장한다. 이 수치는 서버의 HIT/MISS 소요 시간 로그로 측정한 값이다.

```
User 입력 + 변형 유형
        |
        ▼  aiCacheRepository.findByInputTextAndTransformationType()
```

```
┌─── Cache HIT ──→ 1~2ms 반환
│
└─── Cache MISS ──→ Gemini API 호출
                      → JSON 파싱·검증
                      → aiCacheRepository.save()  (다음부터 HIT)
```

그림 4-21. AI 캐싱 아키텍처 (AiLabService.java)

Combine 모드에서는 한 가지 미묘한 문제가 있다. 사용자가 변형자를 임의의 순서로 고르든 결과는 동일한데, 선택 순서가 다르면 캐시 키가 달라져 같은 결과를 중복 생성하게 된다. 이를 변형자 리스트 정렬(Sorting)을 통해 해결하였다. 선택 순서와 무관하게 항상 알파벳 오름차순으로 정렬된 일관적인 캐시 키가 생성되어 같은 캐시를 재사용한다.

```
// AiLabService.java : combineModifiers()
// 선택 순서와 무관하게 결정론적 캐시 키 생성
List<String> sorted = request.getModifiers().stream().sorted().toList();
String transformationType = "combine:" + String.join(",", sorted);
// [past, casual] == [casual, past]  →  동일 키 → Cache HIT
```

그림 4-22. 정렬 기반 캐시 키 정규화 (AiLabService.java)

4.4.4 입력 검증과 오류 처리

Gemini 호출 전에 서버가 입력을 검증한다. 빈 문장, 200자 초과, 한글이 없는 문장, 목록에 없는 범주·변형자, 같은 그룹의 변형자 중복(예: 과거+미래)은 400과 영어 안내로 거절한다. Explore 결과는 항상 정확히 3개를 보장하고, AI 응답이 깨지거나 개수가 모자라면 502, 40초 안에 응답이 없으면 504를 반환한다. 앱은 첫 요청(캐시 MISS)의 대기 시간 동안 거북이가 생각하는 로딩 화면을 보여주고 중복 요청을 막는다.

4.4.5 화면 설계 — 모드 구분과 거북이 실험 컨셉

Sentence Lab의 입력 화면은 "기능이 두 개"라는 것이 한눈에 보이도록 설계하였다. 공통 입력 카드 아래에 Explore와 Combine 모드 카드를 나란히 두고, 실행 버튼은 하단에 하나만 둔다. 로딩과 메뉴는 "거북이가 문장으로 실험한다"는 컨셉으로 통일하였다. 변형은 AI(Gemini)가 생성하므로 메뉴 카드에 "AI-powered"를 표기한다.

구성	내용
공통 입력 카드	문장 입력창(200자) + 탭으로 넣는 예문 칩. 두 모드가 같은 입력을 공유
모드 카드 2개	Explore — 한 규칙으로 변형 3개 (시제·높임·부정·감정 중 하나 선택) / Combine — 여러 규칙을 한 문장에 조합 (그룹마다 하나씩)
실행 버튼	하단 고정 1개, 모드별 라벨(예: Combine 2 rules)

구성	내용
로딩	"Turtle is experimenting with your sentence..." + 단계 문구(Mixing grammar... 등), 실험복 거북이 thinking과 플라스크 장식
Language Lab 메뉴	실험 노트(모눈종이) 무드, 코드로 그린 비커·플라스크, 카드 거품 애니메이션(동작 줄이기 설정 존중). 한글랩 = 장난꾸러기 토끼, Sentence Lab = 모범생 거북이

표 4-9. Sentence Lab 입력 화면과 실험실 메뉴 구성

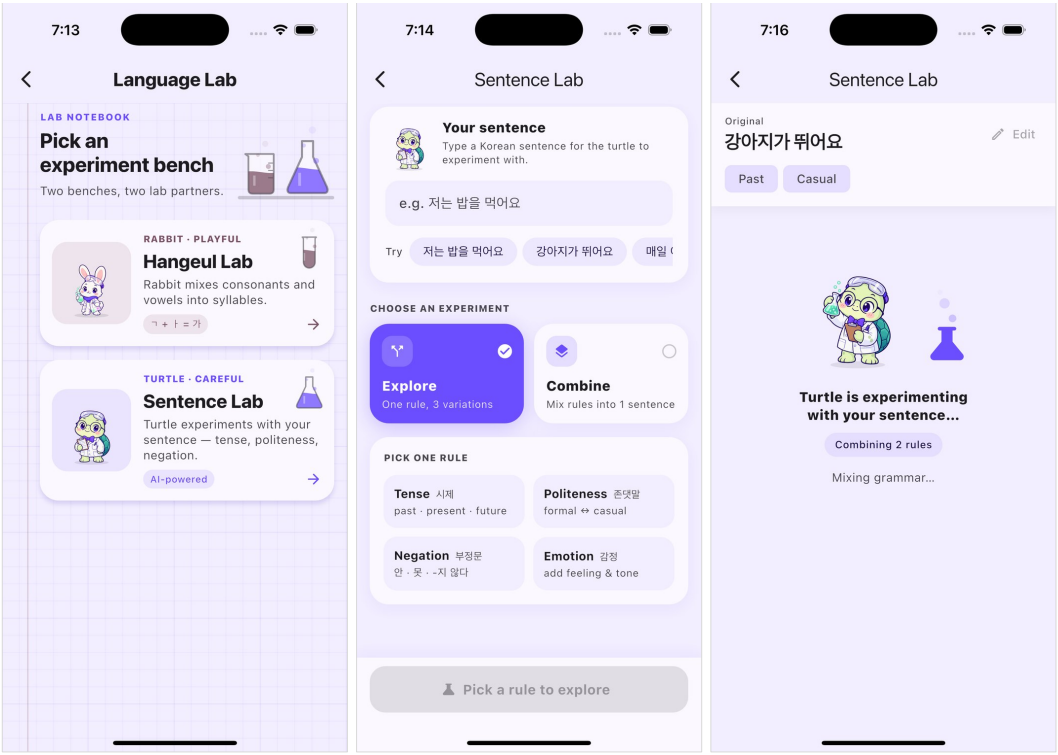


그림 4-23. Language Lab 실험실 메뉴(실험복 캐릭터), Sentence Lab 입력, 거북이 실험 로딩

결과 화면은 입력부를 한 줄 요약으로 접고 원문과 변형 결과를 한 화면에서 비교하도록 구성하였다. 결과 문장은 음절이 아닌 어절 단위로 줄바꿈하고 복사·발음 듣기를 제공한다. 한글랩은 조합 후 주 버튼이 "Try another"(초기화)로 바뀌고, 결과 글자를 서버 TTS로 읽어준다. 한국어 키보드가 없는 학습자를 위해 탭으로 넣을 수 있는 예문 칩을 제공한다.

4.5 캐릭터 시스템 — 토끼와 거북이

토끼와 거북이는 MARU의 학습 철학(1.3절)을 화면에서 보여주는 캐릭터이다. 목표는 학습 이벤트에 살아 반응하는 캐릭터였으며, 전문 애니메이터 없이 짧은 기간에 완성해야 했으므로 "그림은 AI 이미지, 움직임은 코드"로 역할을 나누었다.

구성	내용
표정 (그림)	기본: 토끼·거북이 각 7종(idle·blink·happy·sad·thinking·talking·cheer) + 토끼 magic = 15장. 실험복: 토끼 idle·happy, 거북이 idle·thinking·talking·sad = 6장. 총 PNG 21장. AI로 생성 후 배경 제거·공통 크롭·기준선 정렬·512px 후처리
몸짓 (코드)	Flutter 애니메이션: 숨쉬기·깜빡임·등장 팝인·정답 점프·오답 찌그러짐·cheer 파티클·magic 회전과 반짝임. 접근성 설정(동작 줄이기) 시 정지
공개 API	MaruCharacter(kind, mood, size, reactionKey, outfit) · MaruCharacterBubble(말풍선·타자 효과). 기분(mood)만 바꾸면 표정과 몸짓이 함께 바뀜
폴백	표정 이미지가 없으면 idle 이미지 + 해당 모션, 이미지 자체가 없으면 코드로 그린 플레이스홀더 → 그림 교체 시 기능 코드 수정 0줄
의상	MaruOutfit { normal, lab } — 실험실 화면만 outfit: lab. 실험복 이미지가 없는 표정은 기본 의상으로 자동 폴백(예: 실험복 토끼 sad → 기본 토끼 sad)
적용 범위	4개 기능 21개 화면 지점 + 홈 인사 (아래 표)

표 4-10. 캐릭터 시스템 구성

기능	토끼 🐰 (체험·연기)	거북이 🦕 (분석·코칭)
Korean Lesson	정답 happy, 듣기 퀴즈 반응, 완료 화면 cheer 점프	조립 문제 오답 시 thinking + 힌트 말풍선
Vocabulary	짝맞추기 정답·오답 반응, Perfect Match cheer	오늘의 복습 요약, 복습할 단어 없음 안내
Mission Chat	대화 상대 아바타, 입력 중 thinking, 역할 변신 로딩(magic)	레드·옐로카드 교정, 리포트 판정 이유·Tutor's Note
Language Lab (실험복)	한글랩 담당(장난꾸러기 실험), 조합 성공 happy	Sentence Lab 담당(모범생 실험), 실험 중 thinking, 결과 설명 talking, 오류 sad
홈	인사(idle)	인사(idle), All caught up 카드

표 4-11. 기능별 캐릭터 반응



그림 4-24. 캐릭터 갤러리 — 토끼·거북이 표정 그리드 (개발용 단독 실행 화면)

캐릭터 코드는 기능 코드와 분리된 공유 모듈(lib/shared/characters)로 두고, 각 기능 화면은 정해진 명세(어떤 이벤트에 어떤 기분·크기)를 따라 한두 줄로 적용하였다. 미션 미달성 리포트에서는 토끼가 우는 표정 대신 생각하는 표정과 응원 문구를 써서, 실패가 아니라 재도전으로 느껴지게 하였다.

실험실 화면에서는 실험복 의상(outfit: lab)을 입는다. 의상은 공개 API의 선택 파라미터이며 기본값은 normal이다. 실험복 이미지는 6장이고, 없는 표정은 기본 의상 이미지로 자동 대체된다. 토끼는 장난꾸러기 실험가(한글랩), 거북이는 모범생 연구원(Sentence Lab)이라는 성격을 준다.

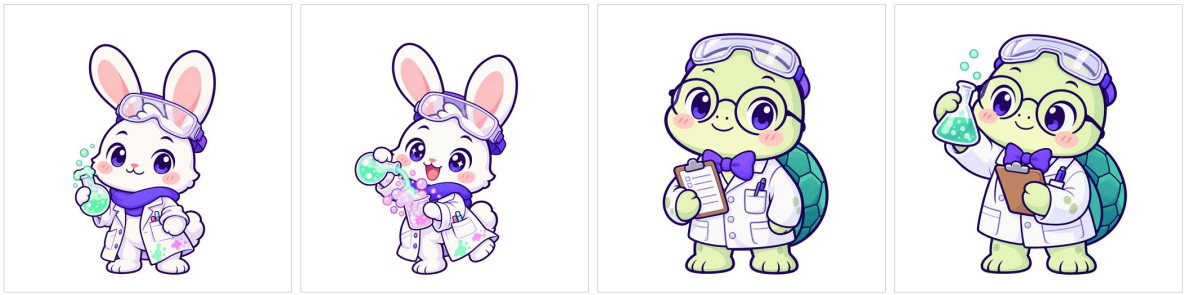


그림 4-25. 실험복 캐릭터 (토끼 idle·happy, 거북이 idle·thinking)

이상으로 MARU의 네 가지 핵심 기능과 이를 하나로 묶는 캐릭터 시스템을 살펴보았다. 네 기능은 모두 추상적인 기술과 규칙을 학습자를 위한 직관적인 학습 경험으로 변환한다는 일관된 철학을 공유한다 — 형태소 분석을 학습 깊이로, 망각곡선을 복습 스케줄로, LLM을 교육학적 궤도로, 유니코드 연산과 AI 캐싱 아키텍처를 자유로운 언어 실험 공간으로 변환하였다.

5. 결과 및 시연

본 장에서는 MARU 앱의 주요 기능별 구현 결과 화면을 제시한다. 화면은 iOS 시뮬레이터(iPhone 16·17 계열)에서 캡처하였으며, 각 기능의 핵심 흐름을 순서대로 확인할 수 있도록 구성하였다. 5.6절에서는 운영 서버와 실제 iPhone에서의 동작을 정리한다.

5.1 로그인, 홈 화면, 학습 통계

앱을 실행하면 Google 또는 Apple 계정으로 로그인한다. 홈에서는 토끼와 거북이가 인사하고, 연속 학습일(스트릭)과 별, 오늘 복습할 단어 수, 네 가지 기능 카드가 표시된다. 복습할 단어가 없으면 All caught up 카드로 바뀐다. Stats 탭은 연속 학습일·별·완료 레슨 수·학습 시간을 보여준다.

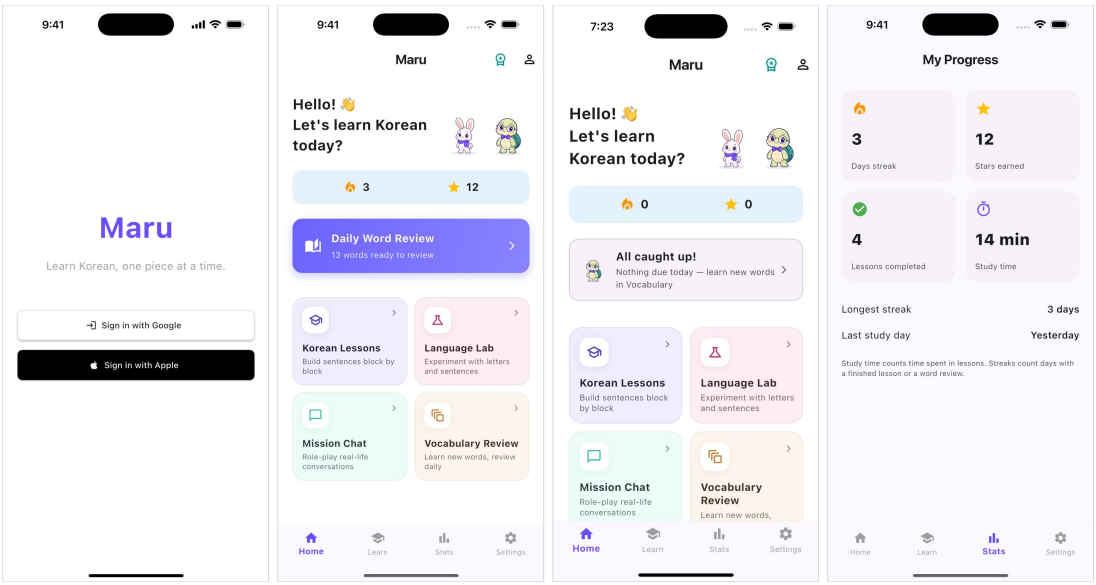


그림 5-1. 로그인(Google·Apple), 홈 화면(복습할 단어가 있을 때 / 없을 때), 학습 통계

5.2 Korean Lesson — 한글 레슨과 형태소 조립

레슨은 유닛별로 구성된다. Unit 0은 한글 12레슨, Unit 1은 문법 3레슨이며, 레슨 목록에는 완료 표

시와 점수에 따른 별이 나타난다. 한글 레슨은 자모 카드와 발음 듣기, 듣기 퀴즈로 진행한다.

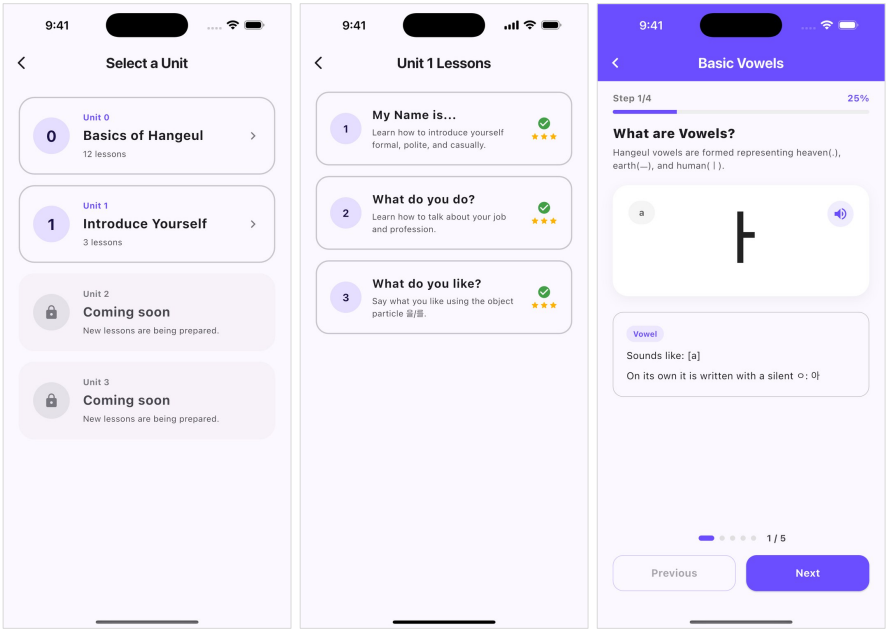


그림 5-2. 유닛 선택, 문법 레슨 목록(완료 표시·별), 한글 자모 카드

문법 레슨은 예문 소개에서 시작한다. 예문의 어절을 탭하면 그 어절을 이루는 형태소와 뜻이 Grammar Analysis 패널에 나타난다(Tap-to-Translate). 예를 들어 "커피를"을 탭하면 커피 = coffee, 를 = object marker로 나뉘어 보인다. 이어지는 패턴 연습에서는 받침 유무에 따라 을/를 중 맞는 형태를 고르고, 토끼가 이유와 함께 정답 여부를 알려준다.

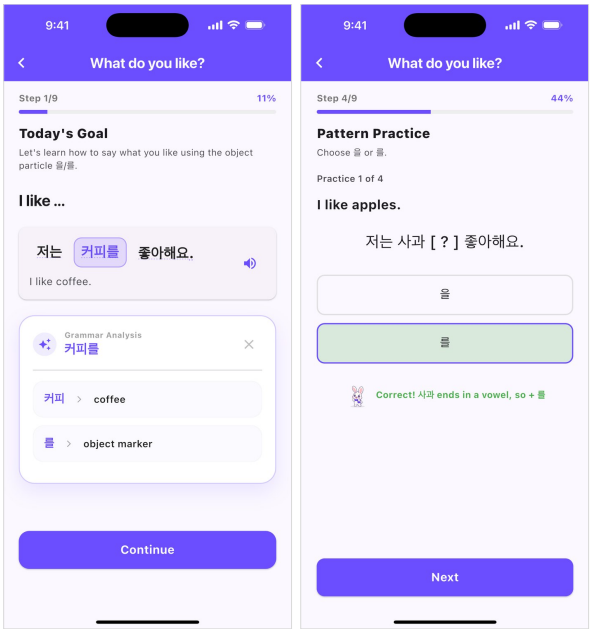


그림 5-3. 탭 분석(커피를 → 커피 + 를), 패턴 연습(을/를 고르기와 토끼 피드백)

조립 문제는 같은 유형의 문장을 두 단계로 조립한다. 🐰 Chunk 단계에서는 의미 덩어리(어절) 블록

으로 문장을 조립하고, 🦕 Morpheme 단계에서는 그 레슨의 목표 문법만 형태소로 분리된 블록으로 다시 조립한다. 목표가 아닌 어절은 회색 고정 블록으로 유지되고, 오답이면 거북이가 힌트를 준다. 그림 5-4는 이 "덩어리 → 목표 문법만 분해" 흐름을 나란히 보여준다. 완료 화면은 학습한 내용을 정리한다.

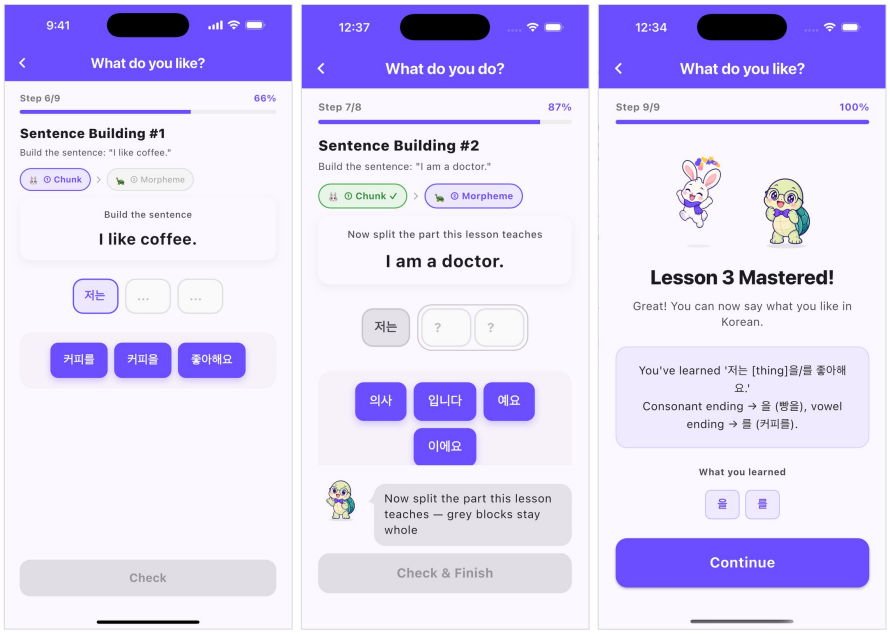


그림 5-4. 🐰 Chunk 단계(덩어리 조립) → 🦕 Morpheme 단계(목표 문법만 분해, 다른 레슨 문장) → 레슨 완료

5.3 Vocabulary System — FSRS 간격 반복 단어 학습

단어장은 14개 주제 덱으로 구성되며, 덱마다 진행률이 표시된다. 단어 카드를 뒤집어 뜻과 예문을 확인한 뒤 Again·Hard·Good·Easy로 평가하면, 버튼 아래에 FSRS가 계산한 다음 복습 간격이 표시된

다.

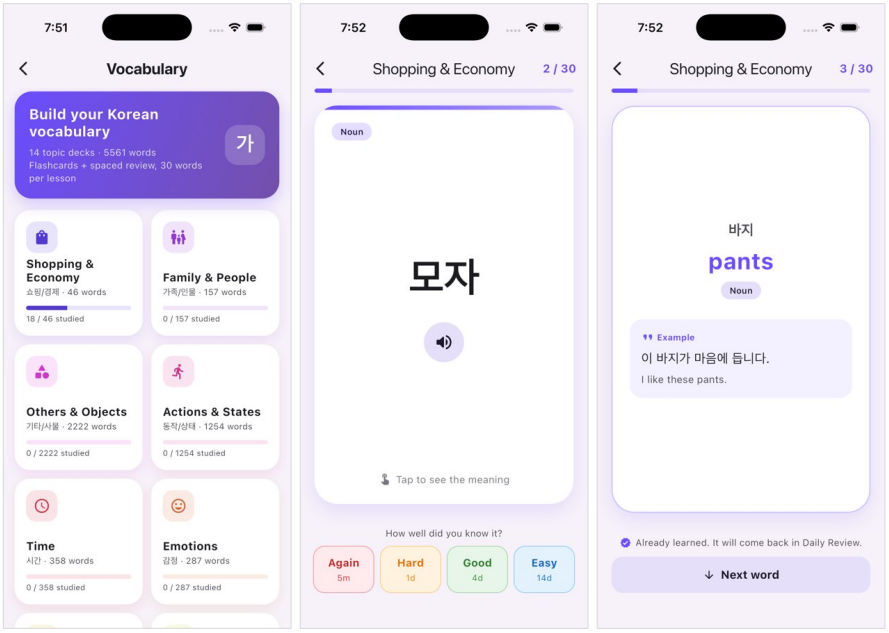


그림 5-5. 덱 목록, 단어 카드와 평가 버튼(다음 간격 5m·1d·4d·14d), 카드 뒷면

짝맞추기 게임은 라운드당 5쌍의 한글 단어와 영어 뜻을 맞춘다. 오늘의 복습은 FSRS 복습 기한이 된 단어만 제시하며, 끝나면 거북이가 요약을 보여준다.

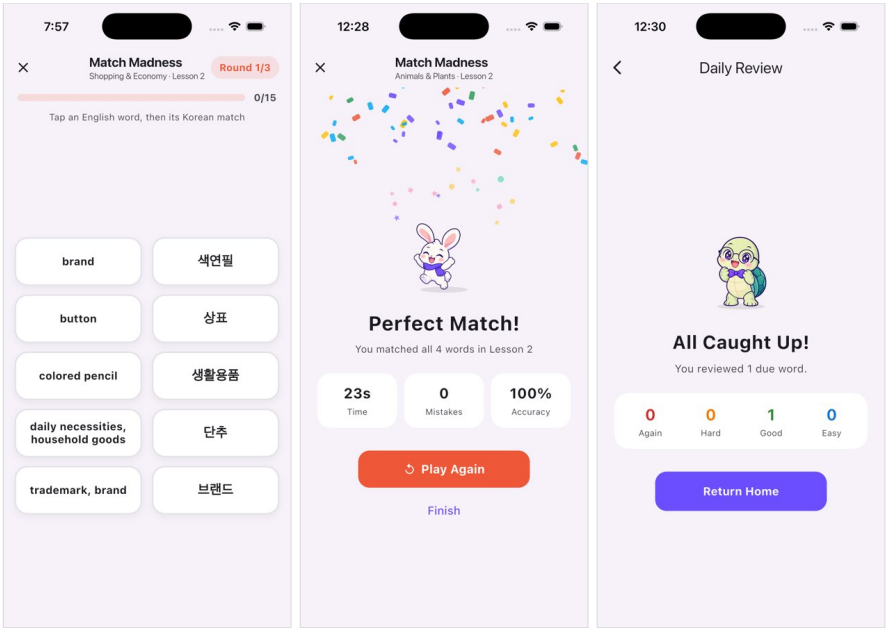


그림 5-6. 짝맞추기 게임, Perfect Match 완료, 오늘의 복습 완료 요약

5.4 Mission Chat — LLM 역할극 미션 대화

미션 설정 화면에서 난이도, 상대와의 관계, 친밀도, 상대 역할을 고르면 토끼가 그 역할로 변신하는

로딩 뒤 대화가 시작된다. 상단 패널에는 미션 제목·난이도·턴 수·목표가 표시된다.

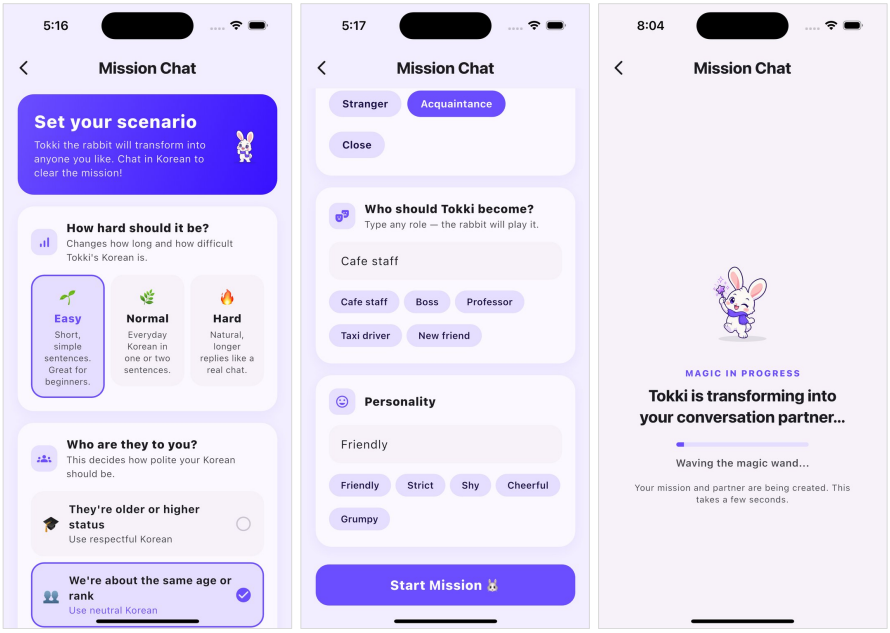


그림 5-7. 미션 설정(난이도·관계), 역할 입력, 토끼 변신 로딩

대화 중 거북이는 작은 실수를 옐로카드로, 대화가 성립하지 않는 큰 실수를 레드카드(해당 문장은 전송되지 않음)로 알려준다. 막히면 Help me Turtle로 힌트 문장을 받을 수 있다.

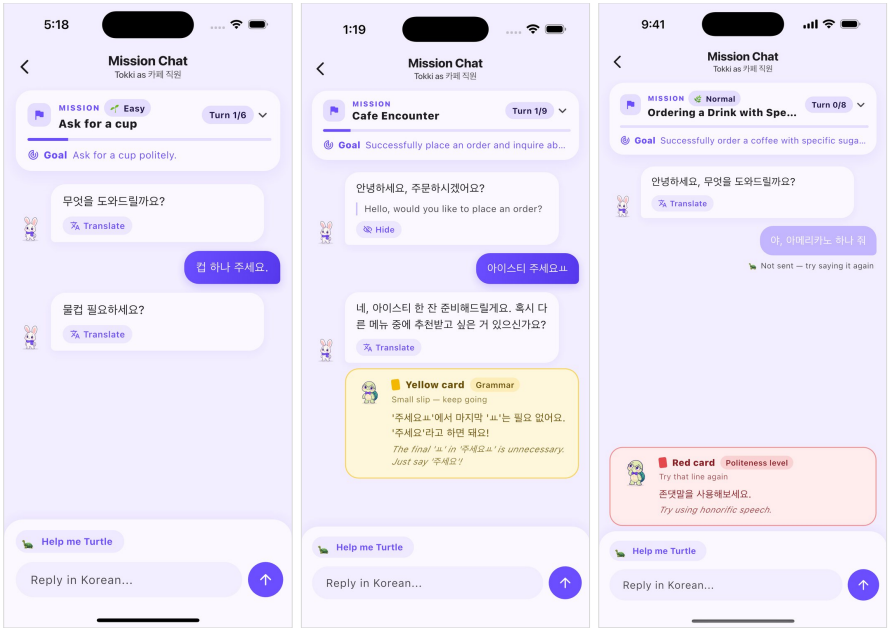


그림 5-8. Easy 난이도 대화, 옐로카드 교정, 레드카드(Not sent)

미션이 끝나면 리포트가 목표 달성 여부와 판정 이유, 잘한 표현·고칠 표현·다음 목표를 보여준다. 달

성하지 못하면 재도전을 권한다. 지난 리포트는 목록에서 다시 볼 수 있다.

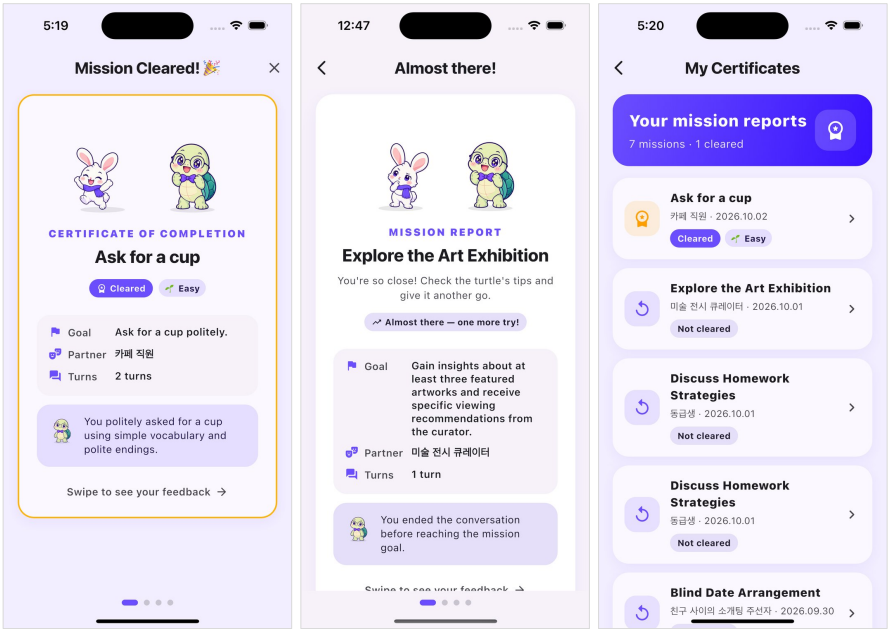


그림 5-9. 미션 달성 리포트, 미달성 리포트(Almost there!), 리포트 목록

5.5 Language Lab — 한글 조합과 Sentence Lab

Language Lab 메뉴에서는 실험복을 입은 토끼(Hangeul Lab)와 거북이(Sentence Lab)가 맞이한다. Hangeul Lab은 초성·중성·종성을 고르면 기기 안에서 바로 글자를 조합하고 발음을 들려준다.

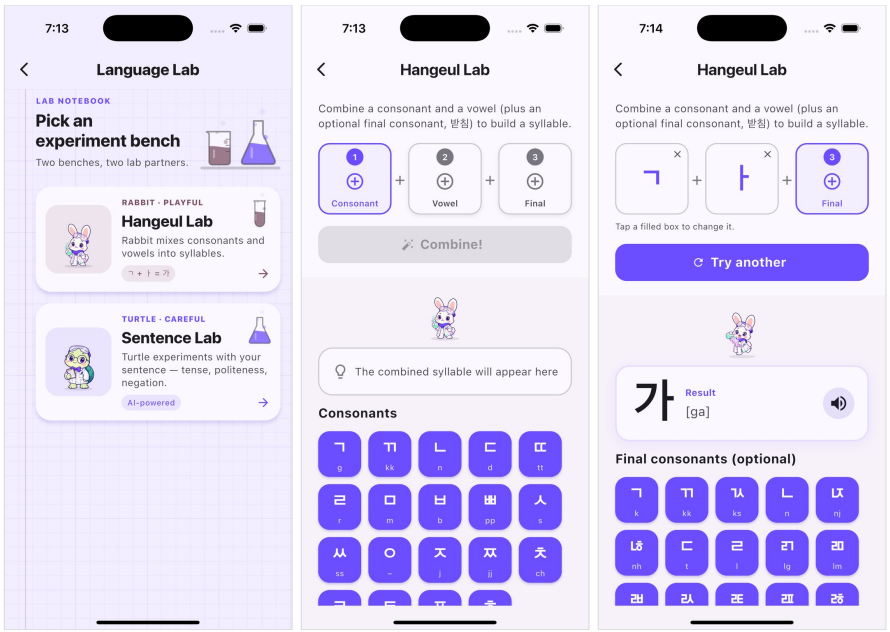


그림 5-10. Language Lab 메뉴, Hangeul Lab 자모 선택, 조합 결과

Sentence Lab은 문장을 입력하고 Explore(한 규칙으로 변형 3개) 또는 Combine(여러 규칙을 한 문장

에 조합)을 고른다. 거북이가 실험하는 로딩 뒤 원문과 변형 결과, 설명이 한 화면에 표시된다.

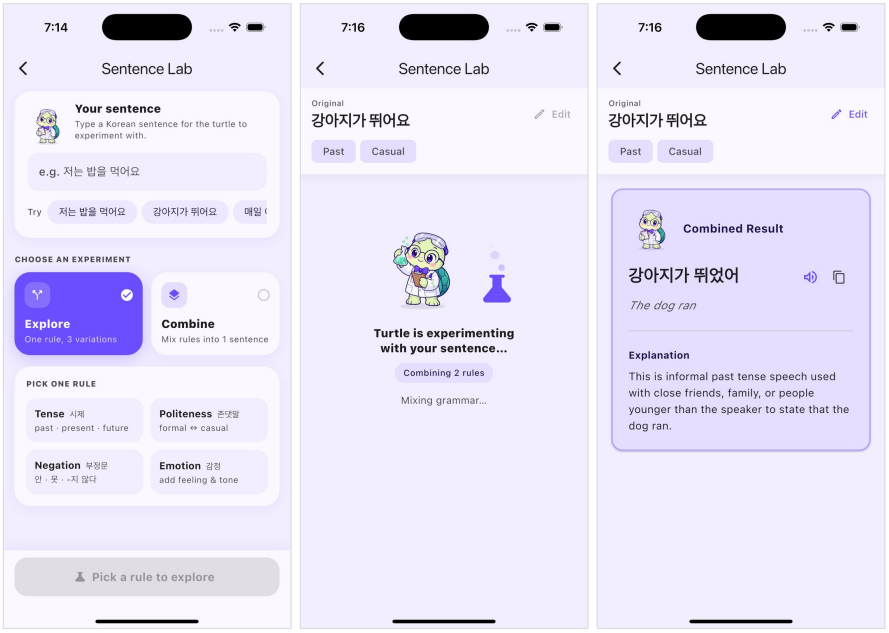


그림 5-11. Sentence Lab 입력(Explore/Combine), 거북이 실험 로딩, Combine 결과

5.6 운영 배포와 실기기 동작

백엔드와 데이터베이스는 Railway에 배포되어 있으며(6.5절), 앱은 빌드 시 서버 주소를 지정한다. 운영 빌드를 설치한 실제 iPhone에서 Google 로그인에 성공하였고, 운영 서버를 통해 레슨·단어장·미션·실험실 기능을 사용할 수 있다.

항목	내용
운영 서버	https://maru2026-production.up.railway.app (GitHub main push 시 자동 배포, 약 2분)
운영 데이터	레슨 15개(한글 12·문법 3), 덱 14개, 단어 5,561개, AI 캐시, TTS 캐시
실기기	iPhone(iOS) 운영 서버 연결 release 빌드, Google 로그인 확인
스모크 테스트	유닛별 레슨 수, 인증이 필요한 API의 403, 잘못된 Google 토큰의 401, 덱 목록 200, 미션 생성 API의 기본 난이도(easy)·기준 턴 응답 확인

표 5-1. 운영 배포 현황

5.7 구현 결과 종합

위 시연 화면들을 통해 MARU의 네 가지 핵심 기능(Korean Lesson, Vocabulary System, Mission Chat, Language Lab)이 실제 동작하는 것을 확인하였다. 각 기능은 교착어로서의 한국어 구조를 학습 설계에 직접 반영하였으며, 형태소 분석(Kiwi), 간격 반복(FSRS-4.5), 대형 언어 모델(GPT-4o, Gemini), 서버 TTS, 유니코드 한글 조합 규칙이 토끼·거북이 캐릭터와 함께 하나의 일관된 학습 경험

으로 통합되었다. 또한 이 기능들이 개발 환경을 넘어 운영 서버와 실제 iPhone에서 동작함을 확인하였다.

6. 개발 과정과 검증

본 장에서는 MARU를 어떤 방법으로 개발하고 어떻게 검증하였는지를 기술한다. 1인 팀이 네 기능과 캐릭터, 배포를 함께 완성해야 했으므로, 작업을 나누고 충돌 없이 합치는 방법 자체가 중요한 설계 대상이었다.

6.1 병렬 세션 개발 방법론

마무리 개발 단계는 AI 코딩 에이전트(Claude Code) 세션 여러 개를 기능별로 병렬 운영하는 방식으로 진행하였다. 개발자(하도윤)는 PM 세션과 함께 계획·결정·통합을 맡고, 각 기능 세션은 정해진 범위 안에서 구현·테스트·기록을 수행하였다. 핵심은 세션들이 서로의 작업을 망가뜨리지 않도록 작업 공간과 소유권을 완전히 분리하는 것이었다.

장치	내용
역할 분리	PM(계획·머지·배포) + 기능 4개 × 서버/앱 세션 + 캐릭터 팀(팀 PM·위젯 개발·에셋) + 제출물 세션
작업 공간 격리	기능마다 git worktree·브랜치(feat/lesson 등), 서버 포트(8081~8084), DB 복제본(maru_<기능>), iOS 시뮬레이터를 고정 배정
파일 소유권	기능별 소유 파일 목록을 명시하고 공유 파일(보안·설정·홈·pubspec 등)은 PM 전용으로 잠금 → 수정이 필요하면 요청 문서로
계약 우선	서버가 API 응답을 바꾸면 API_CONTRACT 문서를 먼저 갱신하고 앱 세션에 알림. 가능하면 필드 추가로 호환 유지
데이터 변경	DB 수정은 멱등 SQL 패치로만, 스키마는 추가만 → 개발 DB 검증 후 운영 DB에 같은 패치 적용
문서 체계	Phase > Step > Task 계획과 태스크 ID([LSN-1.2.3] 등), 세션별 작업 로그와 인수인계(HANDOFF), 상태판, 결정 기록 — 컨텍스트가 요약돼도 이어서 작업
통합	PM이 기능 브랜치를 main에 머지할 때마다 서버 테스트·정적 분석·앱 테스트를 실행. 총 16차례 머지마다 같은 검증을 반복

표 6-1. 병렬 세션 개발을 위한 장치

이 방식으로 짧은 기간에 네 기능의 완성도 작업, 캐릭터 시스템, 네 차례의 사용자 피드백 반영, 운영 배포를 병행할 수 있었다. 반면 세션이 많아질수록 같은 worktree를 쓰는 짝 세션이 git 인덱스를 공유하는 문제, 시뮬레이터 ID를 명시하지 않으면 다른 세션의 기기를 잡는 문제처럼 조율 비용이 생겼고, 이는 규칙 문서에 교훈으로 기록하여 반복을 막았다.

6.2 검증 원칙 — 주장과 동작의 일치

보고서와 발표에 쓰는 모든 문장은 실제 코드와 측정값으로 확인할 수 있어야 한다는 원칙을 두었다. 이를 위해 코드 전체를 대상으로 주장 문장마다 근거(파일 위치, 로그, 테스트)를 대조하는 점검을 수행하고, 확인된 표현만 모은 단일 목록에서 제출물의 문장을 가져왔다. 성능 수치는 서버에 측정 로그를 넣어 얻은 값만 사용한다.

주장	근거
FSRS-4.5 표준 공식으로 복습일 산출	FsrsAlgorithm 단위 테스트, 기본 파라미터에서의 손계산 검증
미션 한 턴 약 1.5~2초, 순차 대비 30~50% 단축	서버 턴 처리 시간 로그(실측 1.4~2.2초, 순차 추정 2.6~4.1초)
캐시 HIT 1~2ms / MISS 3~5초	실험실 서비스의 HIT/MISS 소요 시간 로그
레슨 목표 문법만 형태소로 분해	레슨 3개의 생성 데이터와 앱 화면 확인(표 4-1)
미션은 최대 minTurns+3 턴 안에 종료	3-Zone 규칙 단위 테스트
리포트는 학습자가 실제 보낸 문장만 인용	인용 검증 단위 테스트

표 6-2. 주요 주장과 근거

6.3 사용자 피드백 반영

기능 수정과 별도로, 개발자가 학습자 입장에서 앱을 직접 끝까지 사용하며 불편한 점을 스크린샷과 함께 정리하는 피드백 라운드를 네 차례 진행하였다. 피드백은 각 기능 계획에 태스크로 등록되어 처리되었다. 다만 이는 개발자 본인의 사용 피드백이며, 외부 학습자를 대상으로 한 사용성 평가는 아직 수행하지 못하였다(7.2.5절).

라운드	시점	주요 피드백	반영 결과
1차	09-30	기기 TTS 음질, 한글 카드 네비게이션 중복, 단어장이 너무 단순함, 짝맞추기 애니메이션, 실험실 결과를 보려면 스크롤 필요	서버 TTS, 네비게이션 통합, 단어장 리디자인, Match 애니메이션, 결과 한 화면
2차	10-01	🦕 단계 라벨이 정답을 노출, 미션 로딩 컨셉, 교정 카드 색 구분, 미션 패널 접기, 채팅방 미감, 우는 토끼, 리포트에 내가 안 쓴 문장, 홈 복습 카드	라벨 제거, 토끼 변신 로딩, 레드/옐로카드, 채팅·설정·리포트 리디자인, 사용자 문장 검증, 복습 카드 상시 표시

라운드	시점	주요 피드백	반영 결과
3차	10-02	미션이 초급자에게 너무 어렵고 토끼 말이김, 문장 실험실 입력 화면에서 두 기능이 구분되지 않음, "AI가 만드는 중" 로딩 문구, 홈 카드 4개가 흰색이라 밋밋함	미션 난이도 Easy/Normal/Hard(기본 Easy·토끼 답 길이 상한), Explore/Combine 모드 카드 재설계, "거북이 실험 중" 로딩, 홈 파스텔 카드
4차	10-02	실험실이 실험실처럼 보이지 않음, 문장 실험 기능의 이름이 컨셉과 맞지 않음	화면 이름 Sentence Lab(문장 실험실), 실험 노트 무드 메뉴, 실험복 토끼·거북이(이미지 6장 추가)

표 6-3. 사용자 피드백 라운드와 반영 결과

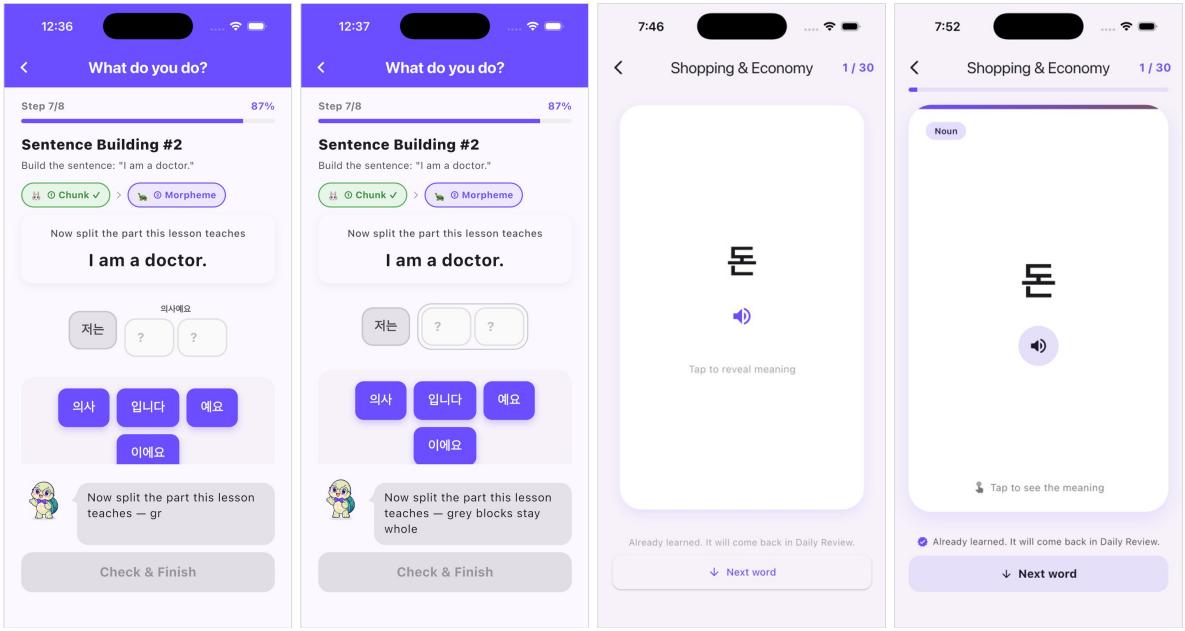


그림 6-1. 피드백 반영 전후 — 🦕 단계 정답 라벨 제거(왼쪽 두 장), 단어 카드 리디자인(오른쪽 두 장)

6.4 자동 테스트와 통합

핵심 로직은 자동 테스트로 검증한다. 서버는 FSRS 공식, 3-Zone 규칙, 난이도 규칙, 리포트 인용 검증, 입력 검증, 오류 응답을, 앱은 캐릭터 위젯과 주요 화면 위젯을 테스트한다. 기능 브랜치를 통합할 때마다 서버 테스트·정적 분석·앱 테스트를 모두 실행하였다.

구분	결과 (최종 통합·운영 배포 기준)
서버 (JUnit, ./gradlew test)	135개 전부 통과
앱 정적 분석 (flutter analyze)	경고 0건
앱 테스트 (flutter test)	83개 통과 (2개 skip)
수동 검증	기능별 시뮬레이터 완주, 서버 curl 확인, 운영 스모크, 실기기 로그인

표 6-4. 자동 테스트 결과

6.5 배포와 데이터 이관

운영 환경은 Railway의 Spring Boot 서비스와 PostgreSQL로 구성된다. GitHub main 브랜치에 반영하면 자동으로 배포되며, 스키마는 서버 기동 시 갱신된다. 콘텐츠는 개발 DB에서 검증한 뒤 콘텐츠 테이블 5개(lessons 15행, word_categories 14행, words 5,561행, ai_cache, tts_cache)만 단일 트랜잭션으로 이관하였다. 개발용 계정·진행 기록·수료증·FSRS 기록은 옮기지 않았다. 이관 후 행 수가 개발 DB와 일치하는지, 시퀀스가 정상인지 확인하고 스모크 테스트를 수행하였다.

배포 절차는 운영 DB 백업 → 필요한 SQL 패치 적용 → main 브랜치 배포 → 스모크 테스트 순서로 고정하였다. 스키마 변경은 추가만 허용하므로, 새 버전 배포 중에도 이전 버전 앱이 그대로 동작한다.

7. 결론 및 향후 계획

7.1 개발 성과 요약

본 프로젝트는 교착어로서의 한국어 구조를 학습 설계에 직접 반영한 모바일 앱 MARU를 Flutter(iOS/Android) + Spring Boot 3.5 + PostgreSQL 기반으로 설계·구현하고, Railway 운영 서버에 배포하였다. 개발 초기에 설정한 네 가지 목표(M-A-R-U)의 달성 여부를 다음과 같이 정리한다.

목표	구현 내용	핵심 기술
M — Morpheme-based Learning	Kiwi 기반 조립 문제 생성 도구로 레슨 목표 문법만 분해하는 2단계(🐰 어절 / 🦕 목표 형태소) 조립 문제 생성. 한글 12레슨 + 문법 3레슨(은/는, 이에요/예요, 을/를), 정답률 점수·별·이어하기, 서버 TTS	kiwipiepy, kiwi-generator, 멱등 SQL 패치, JSONB 레슨 데이터, OpenAI TTS
A — Adaptive Vocabulary	FSRS-4.5 표준 공식·기본 파라미터로 단어별 안정성·난이도·인출 가능성을 계산해 다음 복습일 산출. 오늘의 복습, 다음 간격 미리보기, 5,561단어·14덱, 짝맞추기	FsrsAlgorithm.java, FsrsProgress, Gemini 배치 번역
R — Roleplay Mission	GPT-4o 토끼(연기)·거북이(채점) 병렬 호출로 한 턴 1.5~2초. 난이도 3단계(Easy 기본), 서버 강제 3-Zone, AI 목표 달성 판정, 사용자 문장만 인용하는 리포트, 레드/엘로카드	ChatTurnService, MissionClearanceService, 프롬프트 템플릿
U — Upgrade Your Expression	기기 내 유니코드 한글 조합(Hangeul Lab)과 AI(Gemini) 기반 문장 변형 Sentence Lab(Explore/Combine). 정렬 캐시 키, HIT 1~2ms / MISS 3~5s, 입력 검증	hangeul_combiner.dart, AiLabService, ai_cache
캐릭터	토끼·거북이 이미지 21장(기본 15 + 실험복 6) + 코드 모션, 4개 기능 21개 지점 + 홈	MaruCharacter, MaruCharacterBubble

표 7-1. 목표별 구현 성과

네 가지 목표 모두 실제 동작하는 기능으로 구현되었으며, 형태소 분석·간격 반복 알고리즘·대형 언어 모델·유니코드 처리가 토끼·거북이 캐릭터와 함께 하나의 학습 경험으로 통합되었다. 특히 kiwi-generator 파이프라인은 레슨의 CSV 행과 목표 품사만으로 조립 문제를 생성하고 SQL 패치로 반영하므로, 기존 문제 유형 안에서는 앱·서버 코드를 고치지 않고 콘텐츠를 늘릴 수 있다. 또한 서버 135

개·앱 83개의 자동 테스트와 운영 배포를 통해 개발 환경을 넘어 실제 기기에서 동작하는 서비스의 기반을 갖추었다.

7.2 현재 한계점

구현과 배포를 완료하였으나, 다음과 같은 한계가 남아 있음을 인식하고 있다.

7.2.1 Tap-to-Translate 정적 규칙 문제

현재 Tap-to-Translate 기능에서 어절을 탭할 때 표시되는 형태소별 품사 설명은 generate_morphology.py의 POS_MAP 및 TOKEN_SPECIFIC_MAP에 정적으로 정의된 값을 사용한다. 즉, 동일한 형태소 토큰은 문맥과 관계없이 항상 동일한 설명이 표시된다. 예를 들어 조사 '은/는'은 문장 내 역할(주제 표지 vs. 대조)과 무관하게 고정된 설명을 반환하며, 문맥 의존적 해석이 필요한 경우 오해를 유발할 수 있다. 문맥을 반영한 설명 생성은 향후 과제이다.

7.2.2 Mission Chat 의 난이도 제어와 교정 정확도

LLM 기반 역할극 미션은 시나리오 다양성과 자연스러운 대화 흐름 면에서 강점을 가지나, 다음과 같은 한계가 있다.

첫째, 난이도는 Easy·Normal·Hard 3단계를 학습자가 직접 고르는 방식이다. 학습자의 응답 수준을 실시간으로 감지하여 AI의 언어 복잡도를 자동으로 조절하는 적응형 난이도 제어는 구현하지 않았다.

둘째, 거북이의 교정과 목표 달성 판정의 정확도는 프롬프트에 의존한다. 서버가 3-Zone 규칙·최대 턴·인용 문장 검증으로 형식적 오류는 막지만, 교정 내용이 얼마나 정확한지를 측정할 평가 데이터셋은 아직 없다.

7.2.3 Korean Lesson 콘텐츠 규모

kiwi-generator 파이프라인은 curriculum.csv에 정의된 문장과 Target_POS를 기반으로 퀴즈를 생성하므로, 최종 학습 콘텐츠의 품질은 CSV 데이터의 품질에 직결된다. 현재 문법 레슨은 3개(조립 문제 7개)이고 Unit 2·3은 "Coming soon" 상태로, 서비스 수준의 커리큘럼으로는 충분하지 않다. 소개·연습 단계 JSON은 사람이 작성해야 하며, 문장의 자연스러움·난이도 배열·목표 문법의 다양성 측면에서 교육 전문가와의 협력이 필요하다. 또한 단어 데이터의 번역·예문은 AI가 생성한 것으로 전수 검수가 이루어지지 않았다.

7.2.4 UI/UX 완성도

네 차례의 피드백 라운드로 주요 화면의 디자인과 로딩·오류 상태, 작은 화면 레이아웃을 다듬었다.

그러나 화면 전환 애니메이션의 일관성, Android 실기기 검증, 다크모드·접근성은 아직 충분히 다루지 못하였다.

7.2.5 학습 효과 검증과 알고리즘 개인화

학습 효과에 대한 검증이 가장 큰 한계로 남는다. 현재 사용자는 개발 계정뿐이며, 외부 학습자를 대상으로 한 사용성 평가나 기억 유지율 측정은 수행하지 않았다. FSRS-4.5는 기본 파라미터를 사용하므로 개인의 복습 기록으로 파라미터를 최적화하지 않으며, 공식 구현체(py-fsrs)와의 교차 검증도 하지 못하였다. 덱의 Word Study는 FSRS 추천 순서가 아닌 등급·ID 순 30단어 묶음이고, 조립 문제의 분해 깊이도 학습자 개인의 습득 기록이 아닌 문장별 목표 문법으로 정해진다.

7.3 향후 개발 계획

위의 한계점을 출발점으로 삼아, 향후 MARU의 발전 방향을 단기·중기·장기로 구분하여 제시한다.

구분	개발 항목	내용
단기 (~11월)	웹 체험판·랜딩 페이지	졸업전시(11/6) 관람객이 설치 없이 체험할 수 있는 Flutter Web 체험판(게스트 로그인·비용 제한)과 소개 페이지
단기 (~6개월)	사용성 평가	외부 학습자 대상 사용성 테스트와 설문, 피드백 라운드 정례화
단기 (~6개월)	Tap-to-Translate 동적 설명	LLM 기반 문맥 인식 형태소 설명 생성으로 정적 규칙표 대체, ai_cache 연동으로 비용 최소화
중기 (~12개월)	Korean Lesson 콘텐츠 확장	교육 전문가 협력 커리큘럼(Unit 2·3 포함), 학습한 문법을 새 어휘에 적용하는 전이 학습 단계, 학습자별 습득 기록에 따른 분해 깊이 조절
중기 (~12개월)	FSRS 개인화	복습 기록이 쌓이면 사용자별 파라미터 최적화, py-fsrs 교차 검증, Word Study에 FSRS 큐 적용
중기 (~12개월)	발음 평가 (STT)	음성 인식 기반 발음 피드백, Mission Chat 음성 대화 모드
장기 (~24개월)	다국어 지원 · 학습 분석	일본어·중국어·스페인어 UI 현지화, FSRS·미션·레슨 데이터를 시각화한 학습 분석 대시보드, 학습 효과 측정 연구

표 7-2. 향후 개발 계획

7.4 맺음말

MARU는 '어떻게 하면 교착어인 한국어의 구조를 학습자가 직접 손끝으로 느끼게 할 수 있을까'라는

질문에서 출발하였다. 형태소 분석기, 망각곡선 알고리즘, 대형 언어 모델, 유니코드 조합 규칙이라는 서로 다른 기술들이 하나의 학습 철학 아래 연결될 수 있다는 것을 이 프로젝트를 통해 확인하였다.

물론 완성된 서비스라고 부르기에는 아직 채워야 할 부분이 많다. 콘텐츠의 양과 질, 학습 효과 검증, 알고리즘 개인화 모두 더 나아가야 할 방향이 명확하게 보인다. 그러나 그것이 바로 이 프로젝트가 '졸업 작품의 완성'이 아니라 '서비스의 시작'임을 의미한다고 생각한다.

이 프로젝트에서 가장 크게 배운 것은 "동작한다"와 "설명한 대로 동작한다"는 다르다는 점이다. 보고서의 문장 하나하나를 코드와 측정값에 대조하는 과정에서, 화려한 수치보다 검증할 수 있는 사실이 프로젝트를 더 단단하게 만든다는 것을 확인하였다.

마루를 통해 한국어를 배우는 모든 학습자가 자신만의 마루 정상에 오를 수 있도록, 앞으로도 계속 개발해 나갈 것이다.

8. 참고 문헌

단행본 및 학위논문

[1] Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Leipzig: Duncker & Humblot. [번역판: Ebbinghaus, H. (1913). Memory: A Contribution to Experimental Psychology (H. A. Ruger & C. E. Bussenius, Trans.). New York: Teachers College, Columbia University.]

[2] Woźniak, P. (1990). Optimization of Learning [Master's thesis]. Poznań University of Technology.

학술지 논문

[3] Kim, S. (2024). Towards Robust Morphosyntactic Analysis of L2 Korean: Evaluating and Fine-Tuning a Korean Language Model. ACM Transactions on Asian and Low-Resource Language Information Processing (TALIP). https://doi.org/10.1145/3767330

[4] Lee, B. (2021). Kiwi: Developing a Korean Morphological Analyzer Based on Statistical Language Models and Skip-Bigram. Korean Journal of Data & Information Science (KJDH), 1(1), 109–120. https://accesson.kr/kjdh/v.1/1/109/43508

[5] Park, J., & Kim, H. (2024). Development of morphological diversity in second language Korean: An NLP analysis using the Korean Morphological Richness Analyzer 1.0. System: An International Journal of Educational Technology and Applied Linguistics. https://doi.org/10.1016/j.system.2024.103234

[6] Shadiev, R., Liu, T., & Hwang, W. (2021). Gamification in mobile-assisted language learning: a systematic review of Duolingo literature from public release of 2012 to early 2020. Computer Assisted Language Learning, 35(9), 2412–2435. https://doi.org/10.1080/09588221.2021.1933540

[7] Hwang, G.-J., et al. (2024). LLM-Based Chatbots in Language Learning. ERIC, EJ1455392. https://files.eric.ed.gov/fulltext/EJ1455392.pdf

학술대회 논문

[8] Ye, J., Su, J., & Cao, Y. (2022). A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), 4381–4390. https://doi.org/10.1145/3534678.3539081

[9] Hallifax, S., Serna, A., Marty, J.-C., & Lavoué, E. (2022). When Gamification Spoils Your Learning: A Qualitative Case Study of Gamification Misuse in a Language-Learning App. Proceedings of the 9th ACM Conference on Learning @ Scale (L@S '22). https://doi.org/10.1145/3491140.3528274

기술 보고서 및 기관 자료

[10] OpenAI. (2023). GPT-4 Technical Report. arXiv:2303.08774. https://cdn.openai.com/papers/gpt-4.pdf

[11] Research and Markets. (2024). Korean Language Learning Market — Global Forecast to 2034. Dublin: Research and Markets.

[12] 대한민국 교육부. (2025). 2025년 해외 한국어 교육 현황 조사. 세종: 교육부.

[13] U.S. Foreign Service Institute (FSI). (2023). Foreign Language Training: Language Difficulty Rankings. U.S. Department of State. https://www.state.gov/foreign-language-training/

[14] Korea Tourism Organization (KTO). (2024). 2024년 외래 관광객 입국 통계. 원주: 한국관광공사.

오픈소스 소프트웨어 및 라이브러리

[15] open-spaced-repetition. (2022). free-spaced-repetition-scheduler (FSRS) [Software]. GitHub. https://github.com/open-spaced-repetition/free-spaced-repetition-scheduler

[16] bab2min. (2021). kiwipiepy: Python binding for Kiwi Korean morphological analyzer [Software]. PyPI. https://pypi.org/project/kiwipiepy/

[17] Google. (2023). Gemini API Documentation. Google AI for Developers. https://ai.google.dev/docs

[18] OpenAI. (2023). OpenAI API Reference. https://platform.openai.com/docs

[19] Flutter. (2024). Flutter SDK Documentation. Google. https://flutter.dev/docs

[20] Spring. (2024). Spring Boot 3.5 Reference Documentation. VMware. https://docs.spring.io/spring-boot/docs/current/reference/html/

[21] open-spaced-repetition. (2024). The Algorithm — FSRS-4.5 (fsrs4anki Wiki) [Software documentation]. GitHub. https://github.com/open-spaced-repetition/fsrs4anki/wiki/The-Algorithm

[22] open-spaced-repetition. (2024). py-fsrs: Python package for FSRS [Software]. GitHub. https://github.com/open-spaced-repetition/py-fsrs

[23] OpenAI. (2025). Text to speech — Audio API Guide. https://platform.openai.com/docs/guides/text-to-speech

[24] Railway. (2025). Railway Documentation. https://docs.railway.com